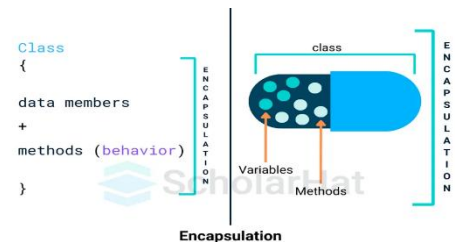


Encapsulation

Encapsulation is defined as the process of binding or wrapping of data and a function in a single unit is called encapsulation.



Abstraction

Abstraction is defined as the process of hiding unnecessary implementation details and showing only the essential information to the user.

Data abstraction is achieved with the different access modifiers.

1. public
2. private
3. protected

1. public: This is the least restrictive access modifier. Classes defined using the public access modifier are accessible across all packages, from derived to unrelated classes.

2. protected: The members of a class defined using the protected access modifier are accessible to
— Classes defined in the same package.
— All derived classes, even if they're defined in separate packages.

3. private: The private access modifier is the most restrictive. The members of a class defined using the private access modifier are accessible only to themselves.

polymorphism

The word **polymorphism** means having **many forms**, and it comes from the Greek words **poly (many)** and **morph (forms)**, this means one entity can take many forms. In Java, polymorphism allows the same method or object to behave differently based on the context.

1. Compile-time Polymorphism (Method Overloading)
2. Run- time Polymorphism (Method Overriding)

Inheritance

The process of acquiring methods and properties from the super class(parent) to sub(child) class by using the keyword **extends** is called inheritance.

Types of inheritance in Java:

1. **Single Inheritance** – One child inherits one parent.
2. **Multilevel Inheritance** – Chain of inheritance (grandparent → parent → child).

3. **Hierarchical Inheritance** – Multiple children inherit from one parent.
4. **Multiple Inheritance** – Java doesn't support class multiple inheritance, but interfaces allow it.

History of Java

- 🚩 Java started in 1991 at **Sun Microsystems**.
- 🚩 It was developed by a small team called the Green Team.
- 🚩 The main creator was **James Gosling** (Father of Java).
- 🚩 The initial name of Java was Oak (named after an oak tree outside Gosling's office)
- 🚩 In **1995**, Oak was renamed to **Java** because the name "Oak" was already taken.
- 🚩 The name "Java" came from **Java coffee**, which the team liked.

Features of JAVA

1. **Simple**
Java is easy to learn because its syntax is similar to C/C++.
This makes coding and debugging much easier for beginners.
2. **Object-Oriented**
Java treats everything as objects, which helps in organizing code.
It supports core OOP concepts: encapsulation, inheritance, polymorphism, and abstraction.
This allows for modular, reusable, and maintainable programs.
3. **Platform-Independent**
Java programs are compiled into **bytecode**, not machine code.
The **JVM (Java Virtual Machine)** executes bytecode on any platform.
This means "write once, run anywhere" is possible.
4. **Secure**
Java provides a secure environment by avoiding pointers and restricting direct memory access.
It includes features like bytecode verification.
5. **Robust**
Java has strong memory management with **automatic garbage collection**.
It supports **exception handling** and strict **type checking** at compile time.
These features help prevent crashes and runtime errors.
6. **Multithreaded**
Java allows multiple threads to run simultaneously within a program.
This is useful for performing concurrent tasks efficiently.
Threads share resources while maintaining proper synchronization.
7. **Portable**
Java programs do not depend on system-specific libraries or hardware.
Compiled bytecode can run on any system with a JVM.
This makes Java applications highly portable across platforms.
8. **High Performance**
Java uses the **Just-In-Time (JIT) compiler** to convert bytecode to native code at runtime.
It optimizes code execution for faster performance.
Although slower than pure compiled languages, JVM optimizations make it efficient.
9. **Distributed**
Java provides built-in support for networking using APIs like **TCP/IP, HTTP, and RMI**.

It allows easy development of distributed applications over the internet.
Objects can communicate across networks seamlessly.

10. **Dynamic**

Java programs can adapt to evolving environments at runtime.

It supports **dynamic loading of classes** and linking at runtime.

This flexibility allows Java applications to grow and integrate with new systems easily.

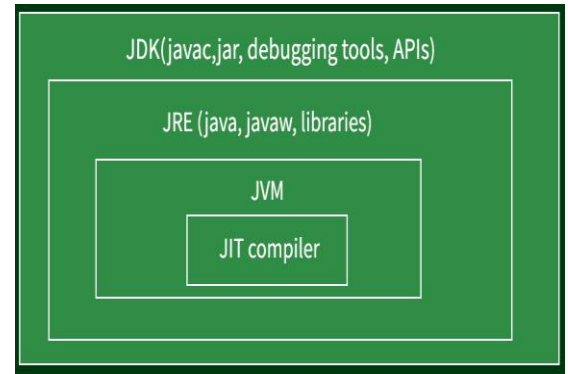
1. JVM (Java Virtual Machine)

JVM is a virtual machine that runs Java programs.

It acts as a platform-independent engine.

Functions of JVM

- ✚ Loads the .class file (bytecode).
- ✚ Verifies bytecode for security.
- ✚ Converts bytecode into machine code.
- ✚ Executes the Java program.
- ✚ Manages memory automatically.
- ✚ Handles exceptions and security checks.



2. JIT Compiler (Just-In-Time Compiler)

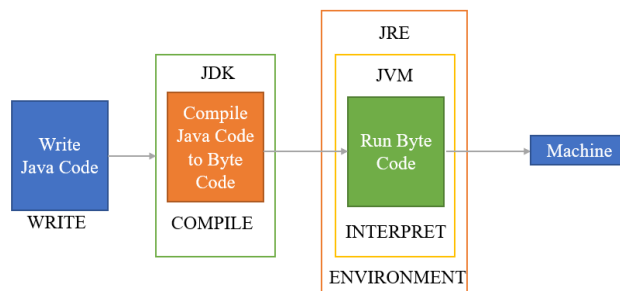
JIT is a part of JVM that converts frequently used bytecode into machine code at runtime. Its main goal is to improve performance.

- ✚ JVM identifies frequently executed code (hot code).
- ✚ JIT compiles that bytecode into machine code.
- ✚ Stores the machine code.
- ✚ Next time, JVM directly runs the machine code → **much faster**.

3. Java Byte Code

Bytecode is platform-independent intermediate code stored in .class files, produced by the Java compiler, and executed by the JVM.

- ✚ You write Java program (.java file).
- ✚ javac (Java compiler) converts it into **bytecode**.
- ✚ JVM loads the .class file.
- ✚ JVM interprets or uses JIT to convert bytecode → machine code



Execution of a simple java program:

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

Save as **Hello.java**

✓ Compile: javac Hello.java

✓ Run: java Hello

The program name should be the class name with the extension .java

Compile: javac program_name.java

Run : java program_name

Keywords in Java

Keywords are reserved words in Java that have a special meaning to the compiler and cannot be used as variable names, class names, or identifiers.

List of Java Keywords

boolean	byte	char	double	float
short	void	int	long	while
for	do	switch	break	continue
case	default	if	else	try
catch	finally	class	abstract	extends
final	import	new	instance of	private
interface	native	public	package	implements
protected	return	static	super	synchronized
this	throw	throws	transient	volatile

Java Comments

Comments are lines in a Java program that are ignored by the compiler. They are used to explain code, improve readability, and help other programmers understand the program.

1. Single-line Comment

- Starts with //
- Used for short explanations.

Ex:

```
// This prints Hello
System.out.println("Hello");
```

2. Multi-line Comment

- Starts with /* and ends with */
- Used for long descriptions.

Example:

```
/* This is a multi-line comment
   It explains the program in detail */
```

3. Documentation Comment

- Starts with /** and ends with */
- Used to generate documentation for classes and methods.

Example:

```
/**
 * This method adds two numbers
 */
```

Data types in java:

A datatype in Java specifies the type of data a variable can store and determines the size, range, and operations that can be performed on that data.

Primitive datatypes

1. Integer Data Types

Data Type	Example	Size	Range (Power Form)
byte	byte a = 10;	1 byte	-2^7 to $(2^7 - 1)$
short	short b = 200;	2 bytes	-2^{15} to $(2^{15} - 1)$
int	int c = 5000;	4 bytes	-2^{31} to $(2^{31} - 1)$
long	long d = 100000L;	8 bytes	-2^{63} to $(2^{63} - 1)$

2. Floating-Point (Decimal) Data Types

Data Type	Example	Size
float	float x = 5.75f;	4 bytes
double	double y = 99.1234;	8 bytes

3. Character Data Type

Data Type	Example	Size	Range (Power Form)
char	char grade = 'A';	2 bytes	0 to $(2^{16} - 1)$

4. Boolean Data Type

Data Type	Example	Range
boolean	boolean pass = true;	true / false

Scope of Variable in Java

The scope of a variable defines the region of the program where the variable is visible and accessible.

- A scope is always defined within a block { }.
- Whenever a new block starts, a new scope is created.
- Variables declared inside a scope cannot be accessed outside that scope.

Major Scopes in Java

1. Method Scope (Local Scope)

- Defined inside a method or block.
- Scope starts from the opening brace { of the method/block.
- Scope ends at the closing brace }.
- Variables are local to the method and cannot be accessed outside it.

2. Class Scope

- Variables declared inside a class but outside methods.

- These are instance or static variables.
- Class scope variables have special properties controlled by access modifiers:
 - public
 - private
 - protected

Program with error

```
class Test {  
    public static void main(String args[]) {  
        int x;  
        x = 10;  
        y = x + 10; // ✗ ERROR: y is not declared (out of scope)  
        x = x + 100;  
        System.out.println("x value " + x);  
    }  
}
```

cannot find symbol

symbol: variable y

location: class Test

Reason

- Variable y is **used without declaration**.
- Hence, y is **out of scope / undefined**.

Implicit / widening type conversion

Implicit type conversion is the automatic conversion of one data type into another by the Java compiler, without using type casting.

It occurs when:

- Source type is smaller
- Destination type is larger
- Conversion is safe (no data loss in range)

Widening Order in Java

byte → short → int → long → float → double

char → int → long → float → double

example:

```
class Test {  
    public static void main(String args[]) {  
        int a = 10;  
        double b = a; // implicit conversion  
        System.out.println(b);  
    }  
}
```

Output :

10.0

Example program for char conversion

```
class Test {  
    public static void main(String args[]) {  
        char ch = 'A';  
        int x = ch;  
        System.out.println(x);  
    }  
}
```

```
}
```

Output : 65

Explicit Type Conversion in Java (Narrowing Conversion)

◆ Definition

Explicit type conversion is the **manual conversion** of one data type into another by the **programmer** using **type casting**.

It is required when:

- Source data type is **larger**
- Target data type is **smaller**
- **Data loss may occur**

double → float → long → int → short → byte
→ char

```
class Test {  
    public static void main(String args[]) {  
        double x = 12.75;  
        int y = (int) x; // explicit conversion  
        System.out.println(y);  
    }  
}
```

Output: 12

Types of Conditional Statements in Java

if Statement

- Executes a block of code only if the condition is true.
- If the condition is false, the block is skipped.

Program:

```
class IfExample {  
    public static void main(String args[]) {  
        int x = 10;  
        if (x > 5) {  
            System.out.println("x is greater than 5");  
        }  
    }  
}
```

if-else Statement

```
class IfElseExample {  
    public static void main(String args[]) {  
        int n = 7;  
        if (n % 2 == 0) {  
            System.out.println("Even number");  
        }  
        Else  
        {  
            System.out.println("Odd number");  
        }  
    }  
}
```

IfElseif conditional statement

```
class IfElseIfExample {  
    public static void main(String args[]) {  
        int marks = 78;  
  
        if (marks >= 90)  
            System.out.println("Grade A");  
        else if (marks >= 75)  
            System.out.println("Grade B");  
        else if (marks >= 50)  
            System.out.println("Grade C");  
        else  
            System.out.println("Fail");  
    }  
}
```

Nested if statement

```
class NestedIfExample {  
    public static void main(String args[]) {  
        int age = 20;  
        int marks = 65;  
        if (age >= 18) {  
            if (marks >= 60) {  
                System.out.println("Eligible for admission");  
            }  
        }  
    }  
}
```

switch Statement – Sample Program

```
class SwitchExample {  
    public static void main(String args[]) {  
        int day = 3;  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            default:  
                System.out.println("Invalid day");  
        }  
    }  
}
```

For loop program:

```
class ForLoopExample {  
    public static void main(String args[]) {  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println("i = " + i);  
        }  
    }  
}
```

Output:

i = 1

i = 2

i = 3

i = 4

i = 5

while loop program:

```
class WhileLoopExample {  
    public static void main(String args[]) {  
  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("i = " + i);  
            i++;  
        }  
    }  
}
```

Output : i = 1
 i = 2
 i = 3
 i = 4
 i = 5

do-while Loop

```
class DoWhileExample {  
    public static void main(String args[]) {  
  
        int i = 1;  
        do {  
            System.out.println("i = " + i);  
            i++;  
        }  
        while (i <= 5);  
    }  
}
```

Output : i = 1
 i = 2
 i = 3
 i = 4
 i = 5

For-Each Loop (Enhanced For Loop) in Java

The for-each loop (also called enhanced for loop) is a simplified version of the for loop designed to iterate over arrays or collections without using an index.

Syntax

```
for (datatype variable : arrayOrCollection) {  
    // statements  
}
```

Key Features

- No need for an **index variable**
- Iterates automatically from **first element to last element**
- Works with **arrays** and **Java Collections** (ArrayList, HashSet, etc.)

Sample program:

```
class ForEachArrayExample {  
    public static void main(String args[]) {  
  
        int[] arr = {10, 20, 30, 40};  
  
        for (int num : arr) {  
            System.out.println(num);  
        }  
    }  
}
```

Output : 10

20

30

40

Sample example for the Break statement :

```
class BreakExample {  
    public static void main(String args[]) {  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) {  
                break; // loop terminates when i = 3  
            }  
            System.out.println(i);  
        }  
    }  
}
```

```
        System.out.println("Loop ended");
    }
}
```

Output : 1

2

Loop ended

Sample program for the Continue statement:

```
class ContinueForExample {
    public static void main(String args[]) {

        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // skip printing 3
            }
            System.out.println(i);
        }


```

```
        System.out.println("Loop ended");
    }
}
```

Output : 1

2

4

5

Loop ended

Arrays in java :

An array is a collection of homogeneous elements (all of the same data type) stored in contiguous memory locations, accessible by indices.

1 Declaration

- Declares a reference variable for the array.
- No memory is allocated yet.

// Syntax 1 (preferred)

```
int[] numbers;
```

// Syntax 2

```
int numbers[];
```

2 Creation

- Allocates memory for the array using the new keyword.

- Specifies the size (number of elements).
`numbers = new int[5]; // creates an array of size 5`

3 Initialization

- Assign values to array elements **either at the time of declaration or after creation.**

- ◆ **At the time of declaration**

```
int[] numbers = {10, 20, 30, 40, 50};
```

- ◆ **After creation**

```
int[] numbers = new int[5];
```

```
numbers[0] = 10;
```

```
numbers[1] = 20;
```

```
numbers[2] = 30;
```

```
numbers[3] = 40;
```

```
numbers[4] = 50;
```

sample program for 1D array:

```
class MaxElementArray {  
    public static void main(String[] args) {  
        int[] numbers = {10, 25, 5, 40, 30};  
        // Assume the first element is maximum  
        int max = numbers[0];  
        // Traverse the array  
        for (int i = 1; i < numbers.length; i++) {  
            if (numbers[i] > max) {  
                max = numbers[i]; // update max  
            }  
        }  
        System.out.println("Maximum element in the array: " + max);  
    }  
}
```

Maximum element in the array: 40

2D-Array:

A 2D array stores data in two dimensions, allowing elements to be accessed using row and column indices.

1. Syntax of 2D Array

Declaration

```
int a[][];
```

Creation

```
a = new int[rows][columns];
```

Initialization

```
int a[][] = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Program to print a 2D array

```
class TwoDArray {  
    public static void main(String args[]) {  
  
        int a[][] = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        for (int i = 0; i < 3; i++) {  
            for (int j = 0; j < 3; j++) {  
                System.out.print(a[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
1 2 3  
4 5 6  
7 8 9
```

Usage of **new** operator:

Objects are created using the **new** operator, which allocates memory dynamically. A class is a logical template, while an object is a real entity occupying memory.

Syntax

```
ClassName objectName = new ClassName();
```

```
class Student {  
    int sno;  
    int math;  
    int phy;  
    int chem;  
}
```

Each object of Student will have its own copy of variables:

sno

math

phy

chem

Multiple Objects

java

Copy code

```
Student stu1 = new Student();
```

```
Student stu2 = new Student();
```

stu1 and stu2 are different objects

Each occupies separate memory locations

Values stored in one object do not affect the other

👉 There can be any number of students, and for every new student, a new object is created to represent that student.

Methods in class

A method is a block of code defined inside a class that performs a specific task and represents the behaviour of an object during program execution.

General Form of a Method

```
returnType  methodName(parameterList) {  
    // body of the method  
}
```

Method Call

```
objectName.methodName();
```

sample code:

```
class Student {  
    int sno;  
    double math;  
    double phy;  
    double chem;  
    double getAverage() { // method definition  
        return (math + phy + chem) / 3;  
    }  
}  
  
public class MethodDemo {  
    public static void main(String args[]) {  
        double avg;  
        Student stud1 = new Student();  
        stud1.sno = 101;  
        stud1.math = 95;  
        stud1.phy = 80;  
        stud1.chem = 85;  
        avg = stud1.getAverage(); // method call  
        System.out.println("Average marks of student: " + stud1.sno + " is "+avg);  
    }  
}
```

Passing Parameters into Methods

Passing Primitive Values (Pass by Value)

// Program for passing primitive values

```
class Student {  
    int sno;  
    double math;  
    double phy;  
    double chem;
```

```

// method definition with parameters
double getAverage(double m, double p, double c) {
    return (m + p + c) / 3;
}
}

```

```

public class MethodDemo {
    public static void main(String args[]) {

```

```

        double avg;
        Student stud1 = new Student();
        stud1.sno = 101;
        stud1.math = 95;
        stud1.phy = 80;
        stud1.chem = 85;

```

```

// method call with arguments
avg = stud1.getAverage(stud1.math, stud1.phy, stud1.chem);

```

```

System.out.println("Average marks of student: " + stud1.sno + " is " + avg);

```

```

    }}

```

Output:

Average marks of student: 101 is 86.66

Passing Object as Parameter

// Program for passing object as parameter

```

class Student {

```

```

    int sno;
    double math;
    double phy;
    double chem;

```

```

// method definition with object parameter
double getAverage(Student st) {
    return (st.math + st.phy + st.chem) / 3;
}

```

```

}

```

```

public class MethodDemo {

```

```

public static void main(String args[]) {

    double avg;
    Student stud1 = new Student(); // object name changed
    stud1.sno = 101;
    stud1.math = 95;
    stud1.phy = 80;
    stud1.chem = 85;

    // method call with object as argument

    avg = stud1.getAverage(stud1);

    System.out.println("Average marks of student: " + stud1.sno + " is "+ avg);

}

```

Output

Average marks of student: 101 is 86.66

Constructors in java

A constructor is a special method used to initialize objects of a class.

Key characteristics:

- Name is same as the class name
- No return type, not even void
- Called automatically when an object is created
- Can be overloaded (multiple constructors with different parameters)

Types of Constructors

A. Default Constructor (No-Argument Constructor)

- Does not take any parameters
- If we do not define a constructor, compiler provides a default constructor automatically
- Used to initialize default values

Example 1: Simple Default Constructor

```

class ConsDemo {
    // default constructor
    ConsDemo() {
        System.out.println("One new object is created.");
    }
}

```

```
public static void main(String[] args) {  
    ConsDemo obj = new ConsDemo(); // default constructor is called  
}
```

Output: One new object is created.

Example 2: Default Constructor with Member Variables

```
class Demo {  
    int value1;  
    int value2;  
    // default constructor  
    Demo() {  
        value1 = 10;  
        value2 = 20;  
        System.out.println("Inside Constructor");  
    }  
    void display() {  
        System.out.println("Value1 = " + value1);  
        System.out.println("Value2 = " + value2);  
    }  
    public static void main(String args[]) {  
        Demo d1 = new Demo(); // default constructor is called  
        d1.display();        // prints initialized values  
    }  
}
```

Output:

Inside Constructor

Value1 = 10

Value2 = 20

Explanation:

- Constructor initializes value1 and value2 automatically when the object d1 is created.
- No need to assign values manually outside the constructor.

B. Parameterized Constructor

- Takes one or more parameters
- Used to initialize objects with different values
- Allows creating distinct objects with unique values

Example: Parameterized Constructor

```
class Student {  
    int id;  
    String name;  
    // parameterized constructor  
    Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
    void display() {  
        System.out.println(id + " " + name);  
    }  
    public static void main(String args[]) {  
        Student s1 = new Student(101, "Swathi"); // first object  
        Student s2 = new Student(102, "Veena"); // second object  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

101 Swathi

102 Veena

Explanation:

- Two objects are created (s1 and s2) with **different values**.
- Each object's constructor is called **automatically** with parameters.
- display() method prints the initialized values.

Difference between Constructor and Method

Feature	Constructor	Method
Purpose	Used to initialize the state of an object	Used to define the behavior of an object
Name	Must be same as class name	Can have any valid name
Return type	No return type (not even void)	Must have a return type (or void)
Invocation	Called automatically (implicitly) when object is created	Called explicitly using object or class name
Compiler support	Compiler provides a default constructor if none is defined	Compiler does not provide any method
Execution	Executes only once per object creation	Can be executed any number of times

Constructor Overloading (Java)

Constructor overloading is a technique in Java where a class contains more than one constructor, and each constructor has a different parameter list.

Example Program: Constructor Overloading

```
class Student {  
    int id;  
    String name;  
    int age;  
  
    // Constructor with two parameters  
    Student(int i, String n) {  
        id = i;  
        name = n;  
    }  
}
```

```

        age = 0; // default value
    }

    // Constructor with three parameters
    Student(int i, String n, int a) {
        id = i;
        name = n;
        age = a;
    }

    void display() {
        System.out.println(id + " " + name + " " + age);
    }

    public static void main(String args[]) {
        Student s1 = new Student(101, "Swathi");
        Student s2 = new Student(102, "Veena", 25);
        s1.display();
        s2.display();
    }
}

```

Output: 101 Swathi 0
 102 Veena 25

Method Overloading (Java)

Method overloading is a feature in Java in which two or more methods have the same name in the same class, but differ in their parameter list.

```

class Shape {
    // Method to calculate area of rectangle
    void area(double l, double b) {
        System.out.println("Area of rectangle is: " + (l * b));
    }
    // Method to calculate area of square
    void area(double s) {
        System.out.println("Area of square is: " + (s * s));
    }
}

```

```
public static void main(String args[]) {  
    Shape sh = new Shape();  
    sh.area(5.0, 10.5); // calls rectangle area  
    sh.area(10.5);     // calls square area  
}
```

Output: Area of rectangle is: 52.5
 Area of square is: 110.25

this Keyword:

In Java, this is a keyword that refers to the current object, the object whose method or constructor is being executed. It is mainly used to:

- Refer to the current class's instance variables and methods.
- Differentiate between instance variables and local variables when they have the same name.
- Pass the current object as an argument to methods or constructors.
- Call one constructor from another within the same class (using this()).

The **this** keyword can be used to access instance variables and methods of the object on which the method or constructor is being invoked.

Output:

```
class Student {  
    int id;  
    String name;  
    Student(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
    void display() {  
        System.out.println(id + " " + name);  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student(101, "Lohith");  
        s1.display();  
    }  
}
```

1. Static Variable

Definition:

A **static variable** is a variable that is declared using the keyword **static** inside a class but **outside all methods**.

Syntax:

```
static int a;
```

Features:

- Memory is allocated only once.
- Memory is created when the class is loaded.
- Same value is shared by all objects.
- Stored in static memory area.
- Accessed using class name.

```
class Test {  
    static int a = 10;  
    public static void main(String[] args) {  
        System.out.println(Test.a);  
    }  
}
```

2. Static Method

Definition:

A method declared using the **static keyword** is called a **static method**.

Features:

- Can be called **without creating object**
- Can access:
 -  Static variables
 -  Non-static variables (directly)
- Called using **ClassName.method()**

Example:

```
class Demo {  
    static void display() {  
        System.out.println("From Static Method");  
    }  
    public static void main(String[] args) {  
        Demo.display();  
    }  
}
```

3. Static Block

Definition:

A static block is a block of code enclosed in { } and starts with the keyword static.

Purpose:

Used to initialize static variables.

Executes only once when class is loaded.

Example:

```
class Test {  
    static int a;  
    static {  
        a = 10;  
        System.out.println("Static Block Executed");  
    }  
    public static void main(String[] args) {  
        System.out.println(a);  
    }  
}
```

Output: Static Block Executed
10

String in Java

-  Strings are a sequence of characters.
-  In Java, strings are objects of the String class.
-  The Java platform provides the String class to create and manipulate strings.

Syntax for String in Java

1. Using String Literal
String variableName = "value";
2. Using new Keyword
String variableName = new String("value");

String methods:

length() → Returns length of string
charAt() → Returns character at given index
equals() → Compares two strings
toUpperCase() → Converts string to uppercase
toLowerCase() → Converts string to lowercase
concat() → Joins two strings
substring() → Extracts part of string
trim() → Removes extra spaces

example:

```
class StringMethods {  
    public static void main(String[] args) {  
  
        String s1 = " Java Programming ";  
        String s2 = "Language";  
  
        System.out.println("Length: " + s1.length());  
        System.out.println("Character at index 2: " + s1.charAt(2));  
        System.out.println("Equals: " + s1.equals(s2));  
        System.out.println("Uppercase: " + s1.toUpperCase());  
        System.out.println("Lowercase: " + s1.toLowerCase());  
        System.out.println("Concat: " + s1.concat(s2));  
        System.out.println("Substring: " + s1.substring(1, 5));  
        System.out.println("Trim: " + s1.trim());  
    }  
}
```

Output:

Length: 18
Character at index 2: v
Equals: false
Uppercase: JAVA PROGRAMMING
Lowercase: java programming
Concat: Java Programming Language
Substring: ava
Trim: Java Programming

Command Line Arguments in Java

Command line arguments are the values passed to a Java program at the time of execution.

They are stored in the `String[] args` parameter of the `main()` method.

Example:

```
class CommandLine {  
    public static void main(String[] args) {  
        System.out.println("First Argument: " + args[0]);  
        System.out.println("Second Argument: " + args[1]);  
    }  
}
```

Compile:

```
javac CommandLine.java
```

```
java CommandLine Java Programming
```

output:

First Argument: Java

Second Argument: Programming

Inheritance

Inheritance is the process by which one class (child/subclass) inherits the fields and methods of another class (parent/superclass) using the extends keyword.

Purpose of Inheritance:

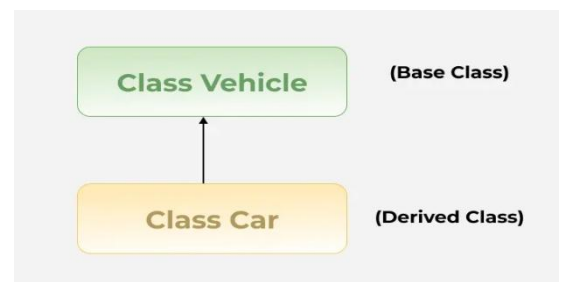
- ✚ Code reusability
- ✚ Method overriding
- ✚ Runtime polymorphism
- ✚ Better code organization

The different types of inheritance which are supported by Java.

- ✚ Single Inheritance
- ✚ Multilevel Inheritance
- ✚ Hierarchical Inheritance
- ✚ Multiple Inheritance
- ✚ Hybrid Inheritance

1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behaviour of a single-parent class. Sometimes, it is also known as simple inheritance.



```
//Super class
class Vehicle {
    Vehicle() {
        System.out.println("This is a Vehicle");
    } }
```

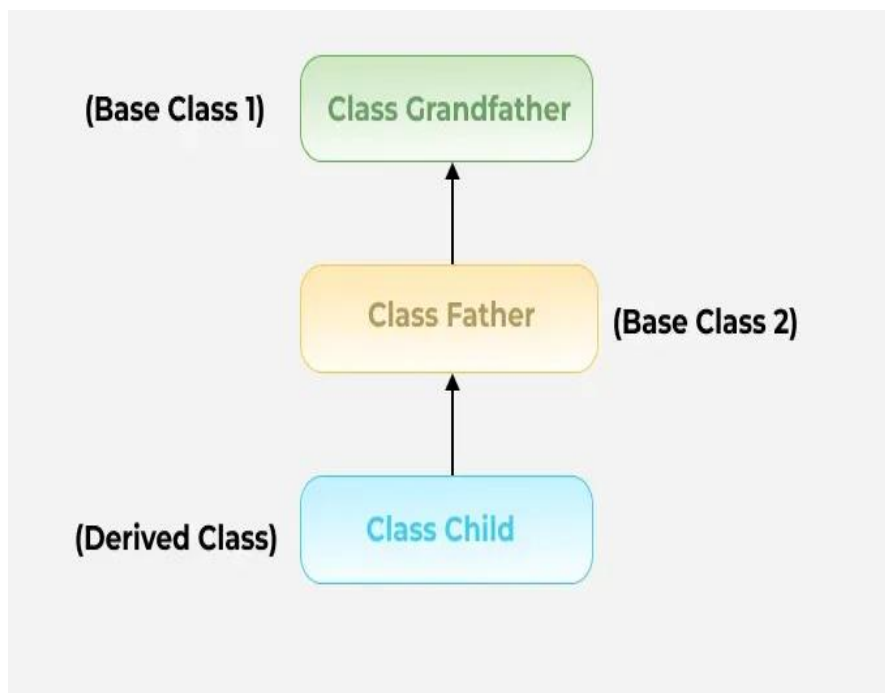
```
// Subclass
class Car extends Vehicle {
    Car() {
        System.out.println("This Vehicle is Car");
    }
}
public class Test {
    public static void main(String[] args) {
        // Creating object of subclass invokes base class constructor
        Car obj = new Car();
    }
}
```

Output

This is a Vehicle
This Vehicle is Car

2. Multilevel Inheritance

Multilevel inheritance allows a **class to inherit properties and methods from a child class, which already inherits from another parent class**



```
class Vehicle {  
    Vehicle() {  
        System.out.println("This is a Vehicle");  
    }  
}  
  
class FourWheeler extends Vehicle {  
    FourWheeler() {  
        System.out.println("4 Wheeler Vehicles");  
    }  
}  
  
class Car extends FourWheeler {  
    Car() {  
        System.out.println("This 4 Wheeler Vehicle is a Car");  
    }  
}  
  
public class Geeks {  
    public static void main(String[] args) {  
        Car obj = new Car(); // Triggers all constructors in order  
    }  
}
```

Output

This is a Vehicle

4 Wheeler Vehicles

This 4 Wheeler Vehicle is a Car

Overriding in Java

The process in which a classes defines a method with the same name, return type, and parameters is called Method Overriding.

- ✚ Method overriding is Runtime polymorphism.
- ✚ Method overloading is Compile time polymorphism.

```

class Parent {
    void show() {
        System.out.println("Parent method");
    }
    class Child extends Parent {
        @Override
        void show() {
            System.out.println("Child method");
        }
    }
    public class Main {
        public static void main(String[] args) {
            Parent obj = new Child();
            obj.show(); // Calls Child's method
        }
    }
}

```

Output: Child method

super Keyword in Java

The super keyword in Java is a reference variable used to refer to the immediate parent class object. It is commonly used to access parent class members (variables, methods, and constructors) from a child class

```

class Parent {
    int num = 100;
}
class Child extends Parent {
    int num = 200;
    void display() {
        System.out.println("Child num: " + num);
        System.out.println("Parent num: " + super.num); // accessing
parent variable
    }
}
public class Main {
    public static void main(String[] args) {

```

```
        Child c = new Child();
        c.display();
    } }
```

Output:

Child num: 200

Parent num: 100

Dynamic Method Dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than at compile time.

It is a key feature of runtime polymorphism in Java.

```
class Parent {
    void show() {
        System.out.println("Parent class method");
    }
}
class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child class method");
    }
}
public class Main {
    public static void main(String[] args) {
        Parent p;    // Parent reference
        p = new Parent();
        p.show();    // Calls Parent's show()
        p = new Child();
        p.show();    // Calls Child's show() → determined at
runtime
    }
}
```

Output:

Parent class method

Child class method

final keyword in java

The final keyword is a non-access modifier used to restrict modification. It applies to variables (value cannot change) methods (cannot be overridden) and classes (cannot be extended). It helps create constants, control inheritance and enforce fixed behaviour.

1. Final Variable

- A variable declared as final **cannot be reassigned** after initialization.
- It essentially becomes a **constant**.

```
class Test {
    final int MAX = 100;
    void display() {
        // MAX = 200; // ✗ Error: cannot assign a value to final variable
        System.out.println(MAX);
    }
}

public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.display();
    }
}
```

Output:

100

2. Final Method

- A method declared as final **cannot be overridden** by subclasses.
- Ensures that the behavior of the method **remains unchanged**.

```
class Parent {
    final void show() {
        System.out.println("Final method in Parent");
    }
}

class Child extends Parent {
    // void show() {} // ✗ Error: Cannot override final method
}

public class Main {
```

```

public static void main(String[] args) {
    Parent p = new Parent();
    p.show();
}
}

```

Output:

Final method in Parent

3. Final Class

- A class declared as final **cannot be subclassed**.
- Useful to prevent inheritance.

```

final class Parent {
    void display() {
        System.out.println("Final class");
    }
}

```

// class Child extends Parent {} // **✗** Error: Cannot inherit from final class

Interface in Java

An interface in Java is a blueprint of a class that contains:

- Abstract methods (methods without body)
- Public static final variables

Methods in Interface

- By default, public and abstract
- No method body

Variables in Interface

- By default, public static final
- Must be initialized

 Java does **not allow multiple inheritance using classes**, but **allows it using interfaces**.

 A class uses **implements** keyword to use an interface.

```

Syntax:    interface InterfaceName    {
                                // variables
                                // abstract methods }

```

Example:

```

interface A {

```

```

    int a = 10; // public static final by default
    void showA(); // public abstract
}
interface B {
    int a = 20;
    void showB();
}
class C implements A, B {
    public void showA() {
        System.out.println("Interface A method");
    }
    public void showB() {
        System.out.println("Interface B method");
    }
    void display() {
        int c = A.a + B.a;
        System.out.println("Sum = " + c);
    }
}
class Test {
    public static void main(String[] args) {
        C obj = new C();
        obj.showA();
        obj.showB();
        obj.display();
    } }

```

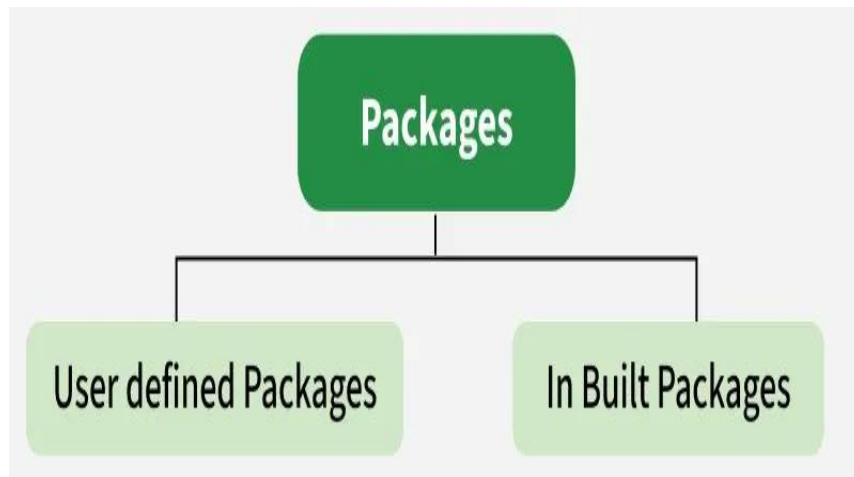
Output: Interface A method
 Interface B method
 Sum = 30

Abstract class Vs Interface

Feature	Abstract Class	Interface
Keyword used	abstract	interface
Methods	Can have abstract & non-abstract methods	Only abstract methods
Method body	Allowed	Not allowed
Variables	Can have normal variables	Only public static final variables
Multiple inheritance	✗ Not supported	☑ Supported
Constructors	✓ Allowed	✗ Not allowed
Access modifiers	Can use private, protected, public	Only public
Inheritance keyword	extends	implements

Java Packages

A package in Java is a mechanism to group related classes, interfaces, and sub-packages into a single unit. Packages help organize large applications, avoid naming conflicts, provide access protection, and make code modular and maintainable.



Built-in Packages in Java

Built-in packages are predefined libraries provided by the Java Development Kit (JDK) that contain a wide range of classes and interfaces to perform common tasks.

These packages help developers avoid writing code from scratch by offering ready-to-use functionalities and are organized under the java

- ✚ **java.lang:** Contains classes and interfaces that are fundamental to the design of the Java programming language. Classes like String, StringBuffer, System, Math, Integer, etc. are part of this package.
- ✚ **java.util:** Contains the collections framework, some internationalization support classes, properties, random number generation classes. Classes like ArrayList, LinkedList, HashMap, Calendar, Date, Time Zone, etc. are part of this package.
- ✚ **java.net:** Provides classes for implementing networking applications. Classes like Authenticator, HTTP Cookie, Socket, URL, URLConnection, URLEncoder, URLDecoder, etc. are part of this package.
- ✚ **java.io:** Provides classes for system input/output operations. Classes like BufferedReader, BufferedWriter, File,

InputStream, OutputStream, PrintStream, Serializable, etc. are part of this package.

- ✚ **java.awt:** Contains classes for creating user interfaces and for painting graphics and images. Classes like Button, Color, Event, Font, Graphics, Image, etc. are part of this package.

User-Defined Packages in Java

User-defined packages are those packages that are designed or created by the developer to categorize classes and packages. It can be imported into other classes and used in the same way as we use built-in packages.

Syntax:

```
package package-name;
```

Import a Specific Class

Imports **only one class** from a package.

Syntax:

```
import packageName.ClassName;
```

Import Entire Package

Imports **all classes** from a package.

Syntax:

```
import packageName.*;
```

program:

```
package mypack;  
  
class MyPackageClass {  
    public static void main(String[] args) {  
        System.out.println("This is my package!");  
    }  
}
```

```
} }
```

>>>> Save the file as **MyPackageClass.java**, and compile it:

```
C:\Users\Your Name>javac MyPackageClass.java
```

Then compile the package:

```
C:\Users\Your Name>javac -d.MyPackageClass.java
```

Sample program to demonstrate user defined packages.

Operations.java

```
package mathpack;
```

```
public class Operations {
```

```
    public int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    public int subtract(int a, int b) {
```

```
        return a - b;
```

```
    }
```

```
    public int multiply(int a, int b) {
```

```
        return a * b;
```

```
    }
```

```
    public double divide(int a, int b) {
```

```
        return (double) a / b;
```

```
    } }
```

>>>> save the file with Operations.java

>>>> compile the file: javac Operations.java

>>> compile the package: javac -d . Operations.java

```
import mathpack.*;
```

```
class TestMath {
```

```
    public static void main(String[] args) {
```

```
        Operations op = new Operations();
```

```
        System.out.println("Addition      : " + op.add(10, 5));
```

```
        System.out.println("Subtraction   : " + op.subtract(10, 5));
```

```
        System.out.println("Multiplication : " + op.multiply(10, 5));
```

```
        System.out.println("Division      : " + op.divide(10, 5));
```

```
    } }
```

Compile and Run

```
javac TestMath.java
```

```
java TestMath
```

Output:

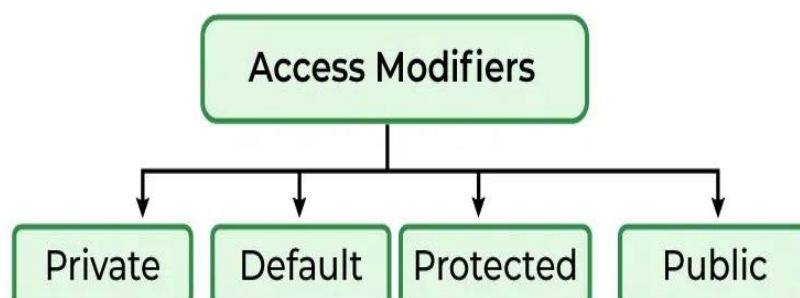
Addition : 15

Subtraction : 5

Multiplication : 50

Division : 2.0

Access Modifiers in Java



Access modifiers are essential tools that define how the members of a class, like variables, methods, and even the class itself, can be accessed from other parts of our program.

Private Access Modifier

The private access modifier is specified using the keyword `private`. The methods or data members declared as private are accessible only within the class in which they are declared.

Default Access Modifier

When no access modifier is specified for a class, method, or data member, it is said to have the default access modifier by default. This means only classes within the same package can access it.

Protected Access Modifier

The protected access modifier is specified using the keyword `protected`. The methods or data members declared as protected are accessible within the same package or subclasses in different packages.

Public Access Modifier

The public access modifier is specified using the keyword `public`. Public members are accessible from everywhere in the program. There is no restriction on the scope of public data members.

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

thank you

- Lekith Kumar

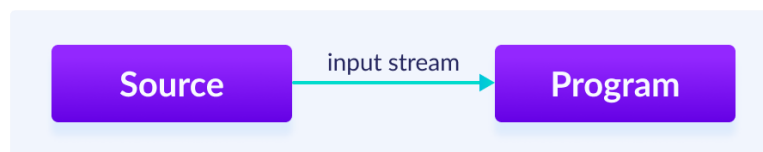
UNIT-3. I/O STREAMS AND COLLECTIONS

Java I/O Streams

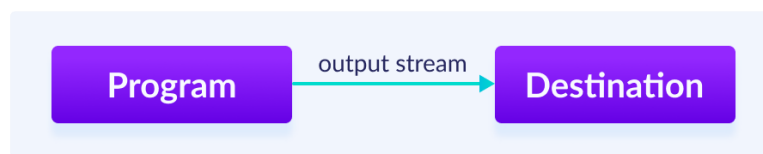
In Java, streams are the sequence of data that are read from the source and written to the destination.

An input stream is used to read data from the source. And, an output stream is used to write data to the destination.

Reading data from source



Writing data to destination



Types of Streams

Depending upon the data a stream holds, it can be classified into:

- Byte Stream
- Character Stream

Byte Stream

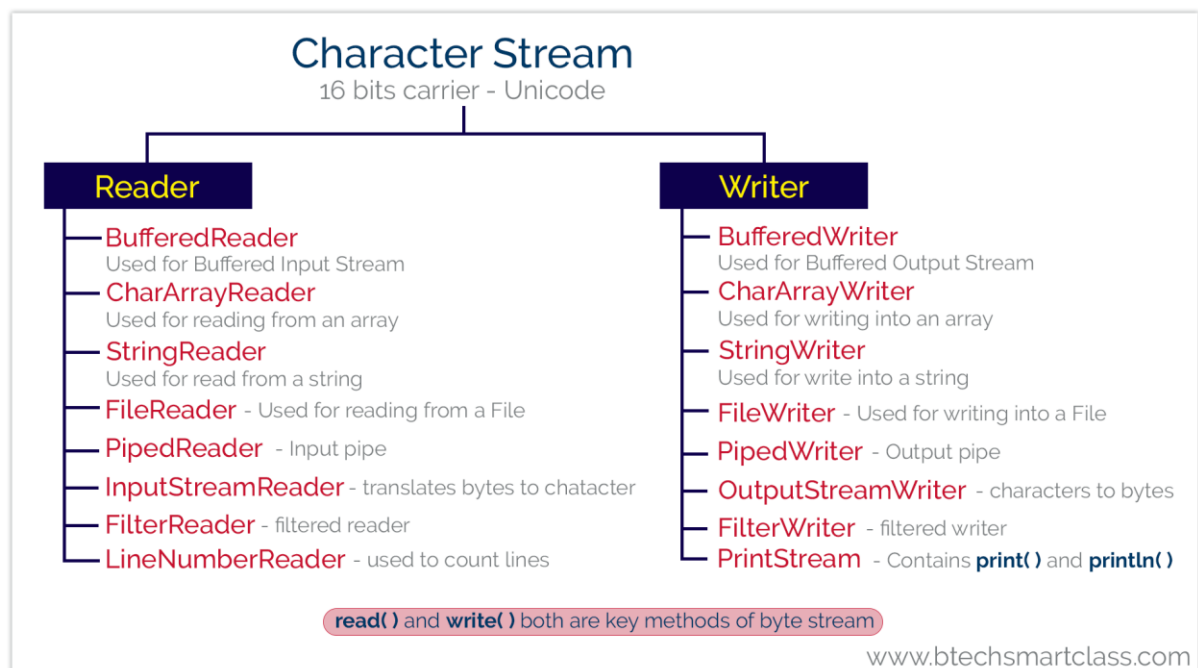
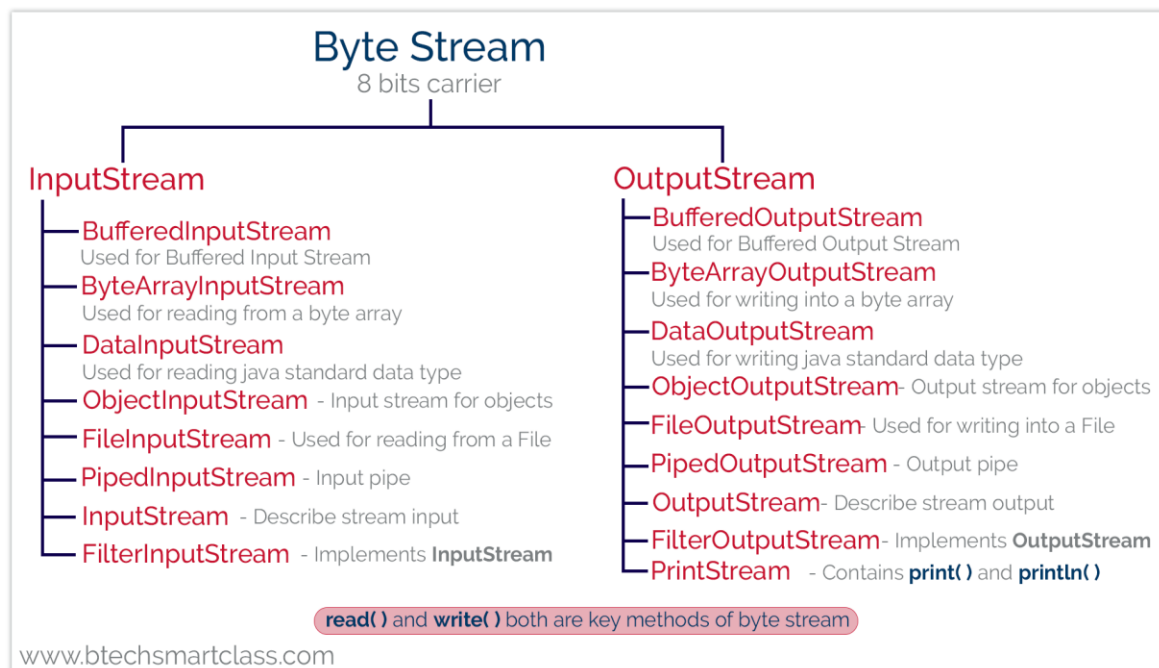
Byte stream is used to read and write a single byte (8 bits) of data.

All byte stream classes are derived from base abstract classes called `InputStream` and `OutputStream`.

Character Stream

Character stream is used to read and write a single character of data.

All the character stream classes are derived from base abstract classes `Reader` and `Writer`.



Reading Console Input in Java (Explanation)

The **java.lang** package defines a class called **System**. This class contains three predefined stream variables:

- **System.in** → Standard **input** stream
- **System.out** → Standard **output** stream

- `System.err` → Standard **error** stream

1. `System.in`

- `System.in` refers to the **standard input stream**
- It is used to **read data from the keyboard**
- Type: **`InputStream`**
- Reads data in **byte format**

2. `System.out` and `System.err`

- Both are objects of **`PrintStream`**
- Used to display output on the console
- **`System.out`**- print normal output message.
- **`System.err`**-prints error output message.

Character-Based Console Input

`System.in` reads **bytes**, Java provides **character streams**.

To obtain a **character-based stream** connected to the console, we wrap `System.in` using:

1. `InputStreamReader` → converts bytes to characters
2. `BufferedReader` → improves efficiency

Syntax to Create `BufferedReader`

`BufferedReader br =`

`new BufferedReader(new InputStreamReader(System.in));`

Reading Characters from Console

`read()` Method

syntax:

`int read()` throws `IOException`

Explanation:

Reads one character at a time

Returns the ASCII value of the character

Returns -1 when end of stream is reached

Can throw IOException

Reading a String from Console

readLine() Method

syntax:

String readLine() throws IOException

Source code:

```
import java.io.*;

class ReadString {

    public static void main(String[] args) throws IOException {

        BufferedReader br =

            new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter your name: ");

        String name = br.readLine();

        System.out.println("Welcome " + name);

    }

}
```

Output:

Enter your name: Praboss

Welcome Praboss

Writing Console Output in Java

Console output in Java is most commonly done using:

- print()
- println()

These methods are defined in the **PrintStream** class.

System.out is an object of **PrintStream**, which is connected to the **standard output device (console)**.

write() Method

The syntax of write() is:

```
void write(int byteval)
```

Explanation:

- Writes **one byte** to the output stream
- Although byteval is an int, **only the lower 8 bits (1 byte)** are written
- Commonly used to print **characters using ASCII values**

```
import java.io.*;

// Demonstrate System.out.write()

public class WriteDemo {

    public static void main(String args[]) throws IOException {

        System.out.write(65); // ASCII value of 'A'

        System.out.write(66); // ASCII value of 'B'

        System.out.write(67); // ASCII value of 'C'

    } }
```

Output- ABC

DataInputStream Class in Java

The DataInputStream class is used to read primitive data types such as int, float, char, double, boolean, etc. from an input stream in a machine-independent way (not just raw bytes).

It is present in the java.io package.

Constructor

DataInputStream(InputStream in)

Creates a DataInputStream that uses the specified input stream.

Methods

1. int readInt()

- Reads 4 bytes and returns an integer value.

2. **byte readByte()**

- Reads 1 byte and returns byte value.

3. **char readChar()**

- Reads 2 bytes and returns a character.

4. **double readDouble()**

- Reads 8 bytes and returns double value.

5. **boolean readBoolean()**

- Reads 1 byte
- Returns **true** if byte is non-zero, otherwise **false**.

6. **float readFloat()**

- Reads 4 bytes and returns float value.

Sample Program:

```
import java.io.*;

public class ReadData {

    public static void main(String[] args) throws IOException {

        DataInputStream dis =

            new DataInputStream(new FileInputStream("number.txt"));

        int num = dis.readInt(); // Reading integer from file

        System.out.println("Number is: " + num);

        dis.close();

    } }
```

→number.txt contains 50

Output: Number is: 50

DataOutputStream Class in Java

The `DataOutputStream` class is used to write primitive data types such as `int`, `float`, `char`, `double`, `boolean`, etc. to an output stream in a machine-independent format (not raw bytes).

It is available in the `java.io` package.

Constructor

DataOutputStream(OutputStream out)

Creates a `DataOutputStream` that writes data to the specified output stream.

-  **writeInt()** → Writes an integer value to the output stream.
-  **writeByte()** → Writes a single byte value to the output stream.
-  **writeChar()** → Writes a character value to the output stream.
-  **writeDouble()** → Writes a double value to the output stream.
-  **writeBoolean()** → Writes a boolean value to the output stream.
-  **writeFloat()** → Writes a float value to the output stream.

```
import java.io.*;
```

```
public class DataOutputDemo {  
    public static void main(String[] args) throws IOException {  
        // Creating FileOutputStream  
        FileOutputStream fl = new FileOutputStream("testout.txt");  
        // Creating DataOutputStream  
        DataOutputStream dout = new DataOutputStream(fl);  
        // Writing data into file  
        dout.writeInt(65);  
        // Saving data  
        dout.flush();  
        // Closing stream  
        dout.close();  
        System.out.println("Success...");  
    }  
}
```

Output: Success...

File Handling in Java (java.io Package)

To handle files in Java programs, the **java.io package** provides the following classes:

1. **FileInputStream class**
2. **FileOutputStream class**

FileInputStream Class

FileInputStream creates an input stream that is used to **read bytes from a file**.

Constructors of FileInputStream

1) FileInputStream(String filepath)

- filepath specifies the **full path name of the file**.

Example:

```
FileInputStream f = new FileInputStream("sample.txt");
```

2) FileInputStream(File obj)

- obj specifies a **File class object** that describes the source file.

Example:

```
File fin = new File("sample.java");
```

```
FileInputStream f1 = new FileInputStream(fin);
```

These constructors throw **FileNotFoundException** if the specified file does not exist.

Methods of FileInputStream

- **read()** → Returns the integer representation of the next available byte; returns **-1** at end of file.
- **available()** → Returns the number of bytes available for reading.
- **close()** → Closes the input stream.

FileOutputStream Class

FileOutputStream creates an output stream that is used to **write bytes to a file**.

Constructors of FileOutputStream

1) FileOutputStream(String filepath)

- filepath specifies the **full path name of the file**.

Example:

```
FileOutputStream f = new FileOutputStream("sample.txt");
```

2) FileOutputStream(File obj)

- obj specifies a **File class object** that describes the output file.

Example:

```
File fout = new File("sample.java");
```

```
FileOutputStream fl = new FileOutputStream(fout);
```

3) FileOutputStream(String filepath, boolean append)

- First parameter → file path
- Second parameter → append mode
 - true → file opened in **append mode**
 - false → file opened in **write mode**

Example:

```
FileOutputStream f = new FileOutputStream("sample.txt", true);
```

📌 All constructors throw **IOException**.

Methods of FileOutputStream

- **write(int b)** → Writes a single byte to the output stream.
- **write(byte[] buf)** → Writes a complete byte array to the output stream.
- **close()** → Closes the output stream.

1. FileOutputStream Program (Writing into File)

```
import java.io.*;
public class FileOutputStreamDemo {
    public static void main(String[] args) throws IOException {
        FileOutputStream fout = new FileOutputStream("sample.txt");
        String data = "Hello Java";
        byte[] b = data.getBytes();
        fout.write(b);
        fout.close();
        System.out.println("Data Written Successfully");
    }
}
```

Output- Data Written Successfully

The file **sample.txt** will contain:

Hello Java

2. FileInputStream Program (Reading from File)

```
import java.io.*;
public class FileInputDemo {
    public static void main(String[] args) throws IOException {
        FileInputStream fin = new FileInputStream("sample.txt");
        int i;
        while((i = fin.read()) != -1) {
            System.out.print((char)i);
        }
        fin.close();
    }
}
```

Output (Console)

Hello Java

Collection Framework in Java

1 What is a Framework?

A **framework** is a set of pre-written classes and interfaces that provide a ready-made architecture to solve common problems.

- It gives reusable design.
- Developers can use it directly without building everything from scratch.
- It improves code reusability and standardization.

In Java, the **Collection Framework** provides a ready structure to store and manage groups of objects.

2 What is Collection?

A **Collection** is a group of individual objects treated as a single unit.

Example:

- List of students
- Set of numbers
- Group of cities

3 What is Collection Framework?

The **Collection Framework** is a set of classes and interfaces in the java.util package used to store and manipulate a group of objects.

Before the Collection Framework:

- Arrays
- Vectors
- HashTables

1. Iterable Interface

- Root interface of Collection Framework.
- Provides iterator() method.
- Allows objects to be used in **for-each loop**.

2. Collection Interface

Oracle Corporation maintains Java, and in Java:

- Collection is the root interface of the Collection Framework.
- It extends Iterable.

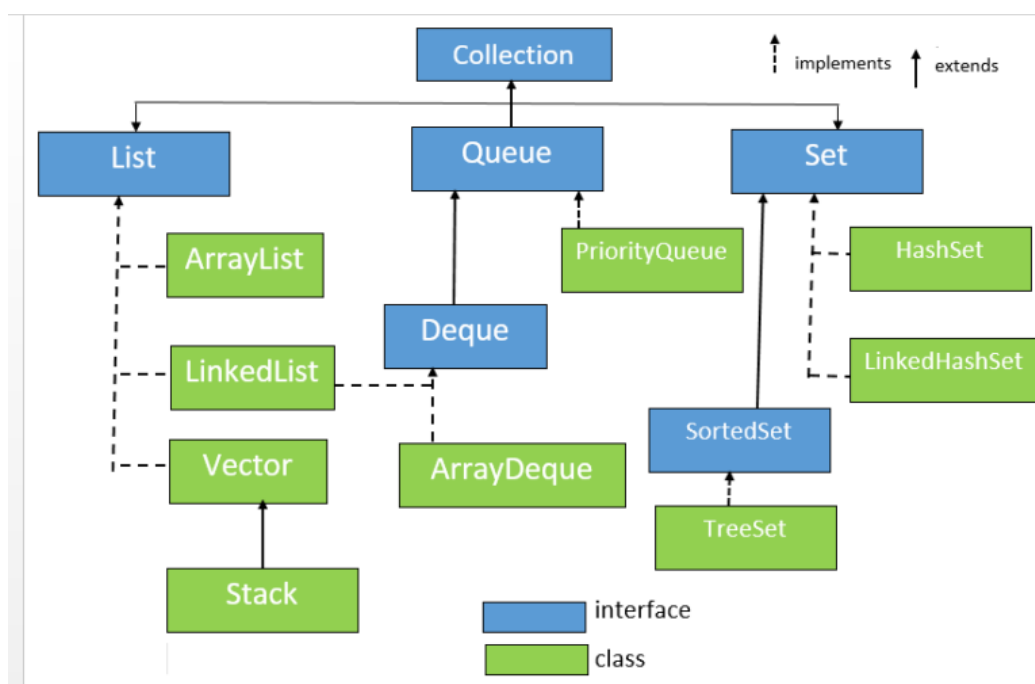
3. Collections Class

- Present in java.util package.
- It is a **utility class**.

Hierarchy of Collection Framework

the **Collection Framework** is present in the java.util package.

It provides a set of interfaces and classes to store and manipulate groups of objects



List of Interfaces under Collection Framework

The main interfaces in the Collection Framework are:

1. Iterable
2. Collection
3. List
4. Queue
5. Set
6. Deque
7. SortedSet

Collection interface

The **Collection interface** is the **root interface of the Java Collection Framework** (except Map).

It is present in the `java.util` package.

It represents a **group of objects as a single unit** and defines the **basic methods** that all collection classes must implement.

It extends the `Iterable` interface, so all collections can be traversed using an iterator or for-each loop.

Methods of Collection Interface

1 boolean add(E e)

Adds an element to the collection.

Returns **true** if the element is successfully added.

2 boolean addAll(Collection<? extends E> c)

Adds all elements of the specified collection to the current collection.

Returns true if the operation is successful.

3 boolean remove(Object o)

Removes the specified element from the collection.

Returns true if the element is removed.

4 boolean removeAll(Collection<?> c)

Removes all elements that are also contained in the specified collection.

5 boolean retainAll(Collection<?> c)

Retains only those elements that are also present in the specified collection.

Removes all other elements.

6 void clear()

Removes all elements from the collection.

7 boolean contains(Object o)

Returns true if the specified element is present in the collection.

8 boolean containsAll(Collection<?> c)

Returns true if the collection contains all elements of the specified collection.

9 int size()

Returns the number of elements in the collection.

10 boolean isEmpty()

Returns true if the collection contains no elements.

List interface

The **List interface** is a child interface of the Collection in the Java Collection Framework.

It represents an **ordered collection of elements** where:

- Duplicate elements are allowed
- Insertion order is maintained
- Elements can be accessed using an index

The List interface is available in the java.util package.

Methods of List Interface

1 boolean add(Object obj)

Adds an element to the end of the list.

2 void add(int index, Object obj)

Inserts the specified element at the given index.

3 Object get(int index)

Returns the element at the specified index.

4 Object set(int index, Object obj)

Replaces the element at the specified index.

Returns the old element.

5 Object remove(int index)

Removes the element at the specified index.

Returns the removed element.

6 boolean remove(Object obj)

Removes the first occurrence of the specified element.

Returns true if the element is removed.

7 int indexOf(Object obj)

Returns the index of the first occurrence of the element.

Returns -1 if not found.

8 int lastIndexOf(Object obj)

Returns the index of the last occurrence of the element.

Set interface

The **Set interface** is a child interface of the Collection in the Java Collection Framework.

It represents a **collection of unique elements**, which means:

- ✗ Duplicate elements are NOT allowed
- ✓ Stores only one copy of each element
- ✓ Does not maintain insertion order (except some implementations)

The Set interface is available in the java.util package.

The **SortedSet interface** is a **subinterface of Set** in the Java Collection Framework.

It represents a **collection of unique elements** that are stored in **sorted (ascending) order**.

1 Object first()

- Returns the **first (lowest)** element in the set.

2 Object last()

- Returns the **last (highest)** element in the set.

3 SortedSet headSet(Object toElement)

- Returns a subset of elements **less than** toElement.

4 SortedSet tailSet(Object fromElement)

- Returns a subset of elements **greater than or equal to** fromElement.

5 SortedSet subSet(Object fromElement, Object toElement)

- Returns a subset of elements **from fromElement (inclusive) to toElement (exclusive)**.

Queue interface

The **Queue interface** is a subinterface of the Collection in Java.

It represents a **collection designed for holding elements prior to processing**, typically in a **FIFO (First-In-First-Out)** order.

- Elements are added at the **rear (tail)** and removed from the **front (head)**.
- Part of the java.util package.
- Can also be used as a **priority queue** or **double-ended queue** using implementing classes.

1 boolean add(Object obj)

- Adds an element (obj) to the **end (rear)** of the queue.

2 Object remove()

- Removes and returns the **front (head)** element of the queue.

3 Object peek()

- Returns the **front (head)** element **without removing** it.

Deque interface

Deque interface (Double-Ended Queue) is a subinterface of **Queue** in Java.

It represents a **collection where elements can be added or removed from both ends – front (head) and rear (tail)**.

1 Object getFirst()

Returns the **first (front)** element of the deque **without removing** it.

2 Object getLast()

Returns the **last (rear)** element of the deque **without removing** it.

3 Object peekFirst()

Returns the **first (front)** element of the deque.

- if the deque is empty, it returns **null**.

Object peekLast()

Returns the **last (rear)** element of the deque.

- if the deque is empty, it returns **null**.

ArrayList

```
import java.util.ArrayList;

public class SimpleArrayList {

    public static void main(String[] args) {

        // Create an ArrayList
        ArrayList<Object> list = new ArrayList<>();

        // Add elements
        list.add("Java");
        list.add("Python");
        list.add(100);

        // Display ArrayList
        System.out.println("ArrayList: " + list);

        // Access element at index 1
        System.out.println("Element at index 1: " + list.get(1));

        // Remove element at index 2
        list.remove(2);

        System.out.println("After removing element at index 2: " +
list);

    } }

ArrayList: [Java, Python, 100]
Element at index 1: Python
```

After removing element at index 2: [Java, Python]

LinkedList

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        // Create a LinkedList
        LinkedList<Object> list = new LinkedList<>();
        // Add elements
        list.add("Java");
        list.add("Python");
        list.add(100);
        list.add("C++");
        // Display the LinkedList
        System.out.println("LinkedList: " + list);
        // Access element at index 2
        System.out.println("Element at index 2: " + list.get(2));
        // Remove element at index 1
        list.remove(1);
        System.out.println("After removing element at index 1: " +
list);
        // Add element at the first position
        list.addFirst("HTML");
        System.out.println("After adding element at first: " + list);
        // Add element at the last position
        list.addLast("CSS");
```

```
        System.out.println("After adding element at last: " + list);  
    }  
}
```

LinkedList: [Java, Python, 100, C++]

Element at index 2: 100

After removing element at index 1: [Java, 100, C++]

After adding element at first: [HTML, Java, 100, C++]

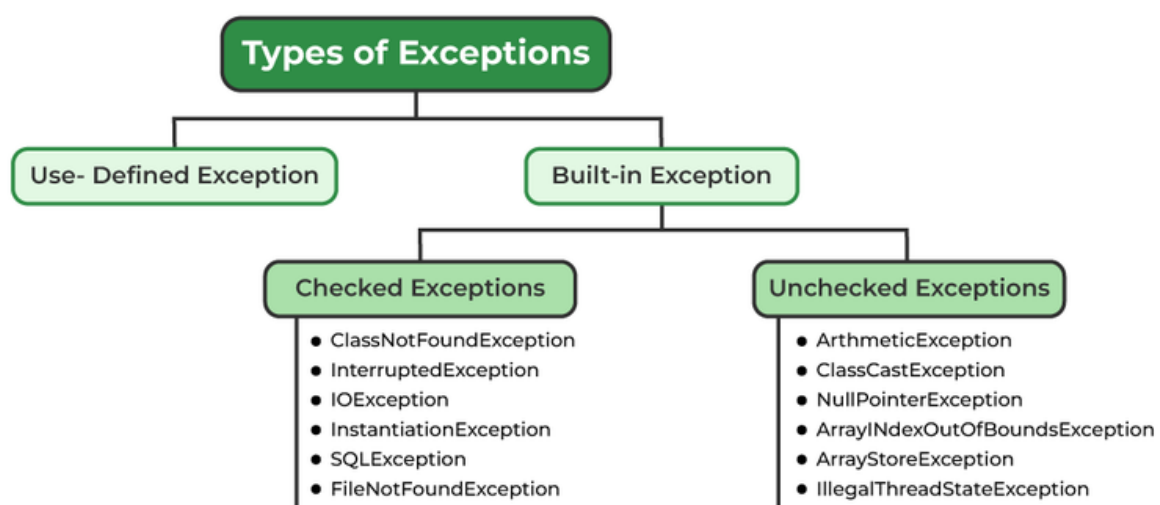
After adding element at last: [HTML, Java, 100, C++, CSS]

4.Exception Handling and Multithreaded Programming

Exception in Java

An exception is a runtime error that occurs during the execution of a program.

- When an exception occurs, the program is abnormally terminated, and the remaining statements are not executed.
- Example: Division by zero → `int x = 10 / 0;`



1 Built-in Exception:

An exception that is predefined by Java and occurs during program execution when a standard error happens,

2 User-defined Exception:

An exception created by the programmer to handle application-specific errors, usually by extending the Exception class.

Checked Exception (also called Compile-time Exception):

A **checked exception** is an exception that is **detected by the compiler at compile-time**.

- ✚ **IOException** – Occurs during input/output operations.
- ✚ **FileNotFoundException** – Occurs when a file being accessed does not exist.
- ✚ **SQLException** – Occurs during database access errors.
- ✚ **ClassNotFoundException** – Occurs when a class is not found at runtime.
- ✚ **InterruptedException** – Occurs when a thread is interrupted.

Unchecked Exception (also called Runtime Exception):

A **runtime exception** that occurs **during program execution** is called an **unchecked exception**.

- These exceptions are **not checked by the compiler at compile-time**.
- The program **compiles successfully**, but may fail during execution.

✚ **ArithmeticException** – Occurs during illegal arithmetic operations, e.g., division by zero.
`int c = 10 / 0;`

✚ **NullPointerException** – Occurs when trying to access or modify a null object.
`String str = null;`
`str.length();`

✚ **ArrayIndexOutOfBoundsException** – Occurs when accessing an array with an invalid index.
`int[] arr = {1,2,3};`
`arr[5];`

✚ **ArrayStoreException** – Occurs when storing an object of the wrong type in an array.
`Object[] arr = new String[3];`
`arr[0] = 10;`

✚ **NegativeArraySizeException** – Occurs when trying to create an array with a negative size.
`int[] arr = new int[-5];`

TO handle these exceptions we require try-catch blocks.

Try Block:

A **try block** is a block of code in Java that **contains statements which may generate an exception** during program execution.

- It is always **followed by one or more catch blocks** to handle the exceptions.
- If an exception occurs in the try block, control is **transferred to the corresponding catch block**.
- If no exception occurs, the catch block is **skipped** and the program continues normally.

Syntax Example (short):

```
try {  
    int c = 10 / 0; // Statement that may cause an exception  
}
```

Catch Block:

A **catch block** in Java is used to **handle exceptions** that occur in the preceding try block.

- It specifies the **type of exception** it can handle.
- When an exception occurs in the try block, control **jumps to the matching catch block**.
- If no exception occurs, the catch block is **skipped**.

Syntax Example (short):

```
catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
}
```

Sample program:

```
public class TryCatchDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            int c = a / b; // May cause ArithmeticException  
            System.out.println("Result: " + c);  
        } catch (Exception e) {  
            System.out.println("Cannot divide by zero");  
        }  
        System.out.println("Program continues after try-catch");  
    }  
}
```

Output: Cannot divide by zero

Program continues after try-catch

Sample program:

```
public class MultipleCatchDemo {  
    public static void main(String[] args) {  
        try {  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[5]);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
            System.out.println(e);  
        }  
    }  
}
```

```
    }  
    catch (ArrayIndexOutOfBoundsException e) {  
        System.out.println("Invalid array index");  
        System.out.println(e);  
    }  
    catch (NullPointerException e) {  
        System.out.println("Null object accessed");  
        System.out.println(e);  
    }  
    System.out.println("Program continues after try-catch blocks");  
} }
```

Output:

Invalid array index

ArrayIndexOutOfBoundsException

Program continues after try-catch blocks

throw Keyword

Definition:

The throw keyword is used to **explicitly throw an exception** in Java.

- It is followed by an **exception object**.
- The exception can be **built-in** or **user-defined**.
- It is used **inside a method or block** to indicate that an error has occurred.

Syntax:

```
throw new ExceptionType("Error message");
```

```
public class ThrowDemo {  
    public static void main(String[] args) {  
        int age = 15;  
        if (age < 18) {  
            throw new ArithmeticException("Age must be at least 18");  
        }  
        System.out.println("Welcome!");  
    }  
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: Age must be at least 18

Finally Block:

A **finally block** in Java is a block of code that **always executes after a try-catch block**, regardless of whether an exception occurs or not.

- It is mainly used to **release resources** like files, database connections, or network streams.
- The **finally block is optional**, but if present, it **executes every time**.

Syntax:

```
finally {  
    // Code that always executes  
}
```

Sample code:

```
public class FinallyDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            int c = a / b; // May cause ArithmeticException  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
        finally {  
            System.out.println("This block always executes");  
        }  
        System.out.println("Program continues normally");  
    }  
}
```

Output: Cannot divide by zero

This block always executes

Program continues normally

throws Keyword

Definition:

- The throws keyword is used in a **method declaration** to **declare the exceptions that a method might throw**.
- It **does not handle the exception**, but **informs the caller** of the method that an exception may occur.

- Used for **checked exceptions** that must be either handled or declared.

Syntax:

```
returnType methodName() throws ExceptionType1, ExceptionType2 {  
    // method code  
}
```

Sample code:

```
import java.io.IOException;  
  
public class ThrowsDemo {  
    static void checkFile() throws IOException {  
        throw new IOException("File not found");  
    }  
  
    public static void main(String[] args) {  
        try {  
            checkFile();  
        } catch (IOException e) {  
            System.out.println(e.getMessage());  
        } } }
```

Output: File not found

Throw vs Throws

Throw	Throws
Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
Throw is followed by an instance.	Throws is followed by class.
Throw is used within the method.	Throws is used with the method signature.
You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

www.geekyshows.com

Introduction to Threads in Java

- A **thread** is the **smallest unit of execution** in a program.
- Java supports **multithreading**, which allows **multiple threads to run concurrently** in a single program.
- Threads are useful for performing **multiple tasks simultaneously**, improving program **performance and responsiveness**.
- Every Java program has at least **one thread**, called the **main thread**, which executes the `main()` method
- All threads share same memory.

Life Cycle of a Thread

A **thread in Java** can be in **one of the following states** during its life cycle:

1. New (Created):

- A thread is created using `Thread t = new Thread();`.

- Thread exists but **has not started** yet.

2. Runnable (Ready to run):

- After calling `t.start()`, the thread moves to **Runnable state**.
- The thread **waits for CPU allocation** to run.

3. Running (Executing):

- When the thread scheduler picks the thread, it **starts executing** the `run()` method.

4. Waiting/Blocked:

- A thread can be **temporarily inactive** while waiting for a resource or another thread
- A thread waits for a **specific period** using methods like

✚ **wait()** – Makes a thread **pause execution** until another thread notifies it.

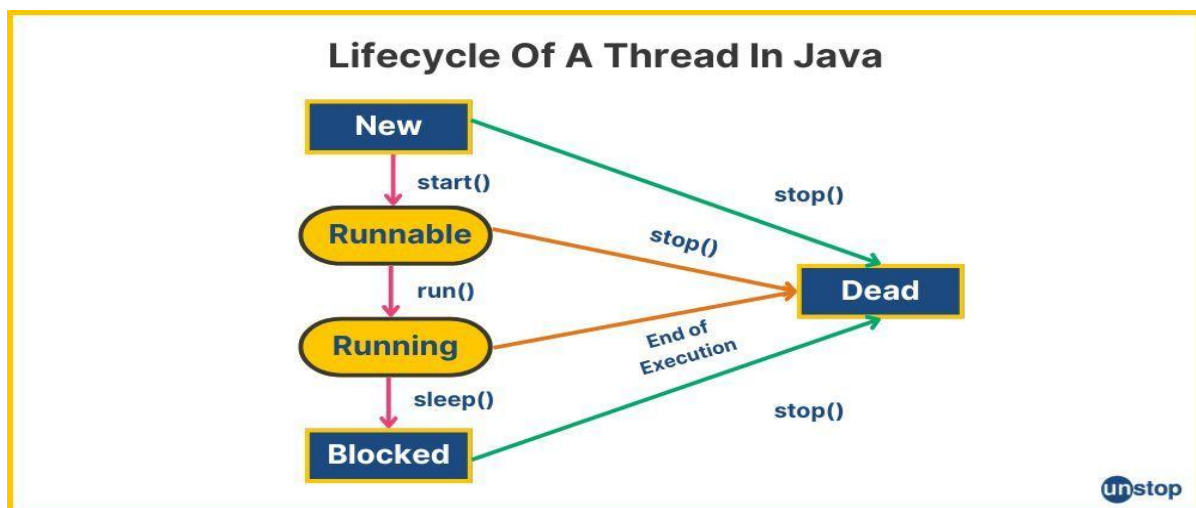
✚ **sleep()** – Makes a thread **pause for a specified time**.

✚ **join()** – Makes the **current thread wait until another thread completes** execution.

5. Terminated/Dead:

- After completing execution of the `run()` method, the thread **dies**.

A dead thread **cannot be restarted**.



Thread Implementations in Java

Java provides **two ways to implement threads**:

- 1** By Extending the Thread Class
- 2** By Implementing the Runnable Interface

By Extending the Thread Class

- Create a class that **extends Thread**.
- **Override** the run() method with the code to execute in the thread.
- Create an object of the class and call **start()** to begin execution.

Sample code:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

```
public class ThreadDemo {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start(); // Start the thread  
    }  
}
```

Output : Thread is running

2 By Implementing the Runnable Interface

- Create a class that **implements Runnable**.
- **Override** the run() method.
- Create a **Thread object** and pass the Runnable object to it, then call **start()**.

```
// Runnable implementation
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running");
    } }

public class RunnableDemo {
    public static void main(String[] args) {
        // Step 1: Create object of Runnable class
        MyRunnable r = new MyRunnable();
        // Step 2: Pass the Runnable object to Thread class
        Thread t1 = new Thread(r);
        // Step 3: Start the thread
        t1.start();
    } }
```

Output: Runnable thread is running.

Thread Priorities in Java

Definition:

- Each thread in Java has a **priority** that **indicates the importance** of the thread.
- Threads with **higher priority are more likely** to be executed before lower-priority threads.
- Thread priority is an **integer value between 1 and 10**.

Method	Purpose
--------	---------

setPriority(int p)	Sets the priority of the thread
--------------------	---------------------------------

getPriority()	Returns the current priority of the thread
---------------	--

Priority Constants in Java

Constant	Value	Description
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Default priority
Thread.MAX_PRIORITY	10	Highest priority

Sample code:

```
class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName() +
            " - Priority: " +
            Thread.currentThread().getPriority());
    }
}

public class ThreadPriorityDemo {
    public static void main(String[] args) {
        // Create threads
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        MyThread t3 = new MyThread();
        // Threads with default priority (NORM_PRIORITY = 5)
        t1.start();
        t2.start();
        t3.start();
        // Change priorities
```

```
t1.setPriority(Thread.MIN_PRIORITY); // 1
t2.setPriority(Thread.NORM_PRIORITY); // 5
t3.setPriority(Thread.MAX_PRIORITY); // 10
System.out.println("Priorities changed after creation.");
System.out.println(t1.getName() + " - " + t1.getPriority());
System.out.println(t2.getName() + " - " + t2.getPriority());
System.out.println(t3.getName() + " - " + t3.getPriority());
}
}
```

Output:

Thread-0 - Priority: 5

Thread-1 - Priority: 5

Thread-2 - Priority: 5

Priorities changed after creation:

Thread-0 - 1

Thread-1 - 5

Thread-2 - 10

Assignment and seminar:

Explain the concept of Synchronization with a sample program.

5.Applets, AWT, Event Handling

Java Applet

An **applet** is a small **Java application program** that is designed to be **transmitted over the Internet and executed by a Java-enabled web browser**.

- It is **dynamically downloaded** across the network similar to **images or sound files**.
- Applets are mainly used to create **interactive web applications**.
- *Applets are deprecated in modern Java versions (Java 9+) and are no longer widely supported by modern web browsers due to security concerns.*

Key Points of Applets

1. Subclass of Applet Class

Every applet is a Java class that **extends the java.applet.Applet class**.

2. Import Statement

All applets must import the package:

```
import java.applet.*;
```

3. No main() Method

- Unlike normal Java programs, **applets do not use main() method**.
- Execution starts from special **lifecycle methods**.

4. Execution Environment

- Applets run inside:
 - a **Java-enabled Web Browser**, or
 - an **Applet Viewer**.

5. Embedded in HTML

- Applets are placed inside an **HTML page**.

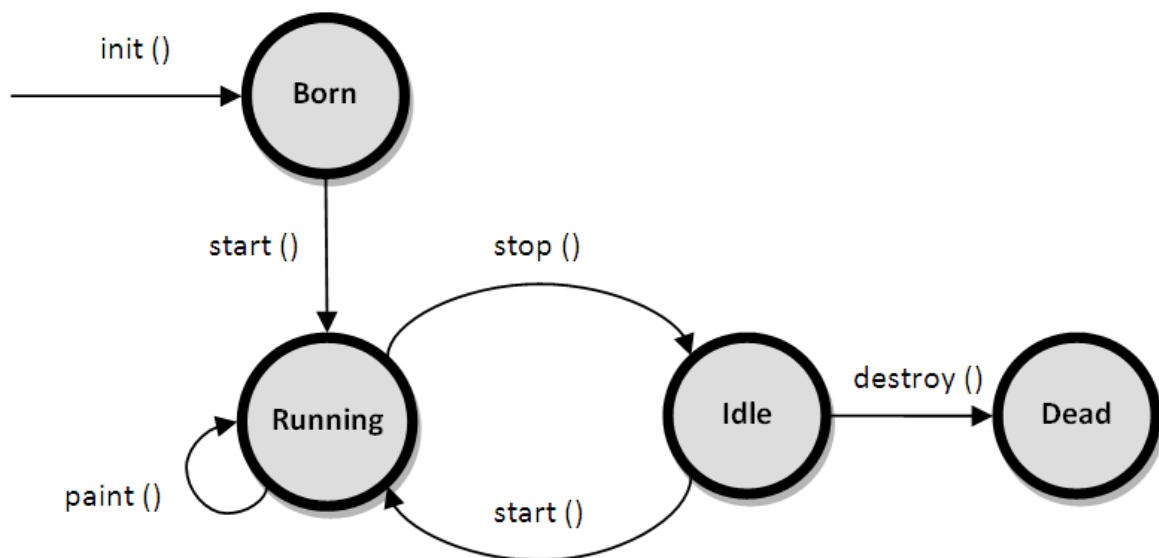
Applet Tag in HTML

```
<applet code="AppletName.class" width="300" height="200">  
</applet>
```

Attributes

- **code** → Name of the applet class file
- **width** → Width of the applet window
- **height** → Height of the applet window

Life cycle of an Applet



The **Applet Life Cycle** consists of **five important methods** that control the execution of an applet. These methods are automatically called by the **web browser or applet viewer**.

The five methods are:

init(), start(), paint(), stop(), destroy()

1. init()

- It is the **first method called** in the applet life cycle.
- Used to **initialize variables and objects**.
- Called **only once** during the runtime of the applet.
- Invoked at the **time of initialization**.

Syntax

```
public void init()  
{  
    // To initialize objects  
}
```

2. start()

- Called **after the init() method**.
- Used to **start the execution of the applet**.
- Also used to **restart the applet** after it has been stopped.

Note:

init() is called **only once**, but start() is called **every time the HTML page containing the applet is displayed**.

Syntax

```
public void start()  
{  
    // To start the applet code
```

}

3. paint()

- Used to **display output and draw shapes** like square, rectangle, circle, etc.
- It contains **one parameter of the Graphics class**.
- The **Graphics object provides the drawing context**.

Note:

This is the **only method among the applet life cycle methods that has parameters**.

Syntax

```
public void paint(Graphics g)
{
    // Drawing code
}
```

Example:

```
g.drawString("Hello Applet", 50, 50);
```

4. stop()

- Called when the **browser leaves the page containing the applet**.
- Invoked when the browser is **minimized, stopped, or switched to another page**.
- Used to **suspend threads or stop processes** that should not run when the applet is not visible.

- The applet can be **resumed again using start()**.

Syntax

```
public void stop()
{
    // To stop the applet code
}
```

5. destroy()

- Called when the applet is **removed completely from memory**.
- Used to **clean up resources**.
- Called **only once**.
- After this method is executed, the applet **cannot be restarted**.

Note:

stop() method is **always called before destroy()**.

Syntax

```
public void destroy()
{
    // To destroy the applet
}
```

Source code:

```
import java.applet.Applet;
import java.awt.Graphics;
public class HelloWorldApplet extends Applet {

    // Overriding the paint() method to draw on the applet
    window

    @Override
    public void paint(Graphics g) {
        g.drawString("Hello World", 50, 50);
    }
}
```

Steps to Compile and Run

1. **Save** the Java code as HelloWorldApplet.java
2. **Compile** the Java code using the javac command in your terminal:

```
javac HelloWorldApplet.java
```

3. **Run** the applet. You can use the appletviewer tool provided by the JDK to view it without a browser (modern browsers have largely deprecated support for Java applets):

```
appletviewer HelloWorldApplet.java
```

OUTPUT:



Graphics Class Methods

update(), paint(), drawing lines, rectangles, circles, polygons

Java **AWT Graphics class** is used to **draw shapes and text in an applet window.**

1.update() Method

The **update() method** is used to **clear the previous drawings on the screen** before new drawings are displayed.

- It **erases the earlier graphics.**
- After clearing the screen, it **calls the paint() method** to redraw the graphics.
- This helps in **refreshing the applet window.**

If programmers call **paint()** directly, they might forget to clear the previous graphics.

So Java internally calls **update() → paint()** to avoid this issue.

Syntax

```
public void update(Graphics g)
{
    // Clears the previous drawings
}
```

2. paint() Method

The **paint()** method is used to **display graphics and text in the applet window**.

It is **automatically called by the browser or applet viewer** whenever the applet needs to be redrawn.

Situations when paint() is called

1. When the **applet window appears for the first time**.
2. When the **window is restored from a minimized state**.
3. When the **window size is changed (resized)**.

Syntax

```
public void paint(Graphics g)
{
    // Drawing code
}
```

The **Graphics object (g)** allows us to **draw shapes, lines, and text**.

Example:

```
g.drawString("Hello Applet", 50, 50);
```

3. Graphics Class

The **Graphics class** is part of **Java AWT (Abstract Window Toolkit)**.

- It is an **abstract class** used for **drawing on GUI components**.
- It contains **methods to draw shapes, text, and images**.
- It **cannot be instantiated directly**.

The **Graphics object** is obtained inside:

- `paint(Graphics g)`
- `update(Graphics g)`

Common Methods of Graphics Class

1. drawString()

Used to **display text in the applet window**.

```
public abstract void drawString(String str, int x, int y)
```

Example:

```
g.drawString("Java Applet", 100, 100);
```

2. drawRect()

Used to **draw a rectangle**.

```
public void drawRect(int x, int y, int width, int height)
```

Example:

```
g.drawRect(50, 50, 100, 60);
```

3. fillRect()

Used to **draw a filled rectangle**.

```
public abstract void fillRect(int x, int y, int width, int height)
```

Example:

```
g.fillRect(70, 70, 100, 50);
```

4. drawOval()

Used to **draw an oval or circle**.

```
public abstract void drawOval(int x, int y, int width, int height)
```

Example:

```
g.drawOval(100, 100, 80, 80);
```

5. fillOval()

Used to **draw a filled oval or circle**.

```
public abstract void fillOval(int x, int y, int width, int height)
```

Example:

```
g.fillOval(120, 120, 60, 60);
```

6. drawLine()

Used to **draw a line between two points**.

```
public abstract void drawLine(int x1, int y1, int x2, int y2)
```

Example:

```
g.drawLine(10, 20, 200, 20);
```

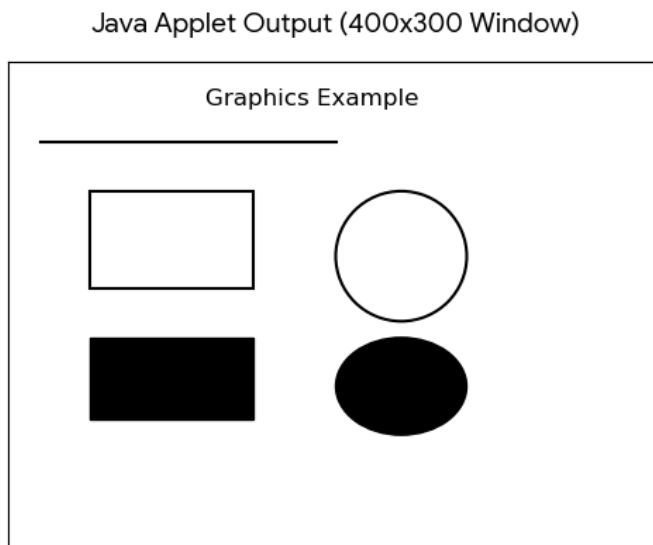
Example Program (Drawing Shapes in Applet)

```
import java.applet.Applet;
```

```
import java.awt.Graphics;
```

```
/*  
  
<applet code="ShapesApplet.class" width="400"  
height="300">  
  
</applet>  
  
*/  
  
public class ShapesApplet extends Applet  
{  
    public void paint(Graphics g)  
    {  
        g.drawString("Graphics Example", 120, 30);  
  
        g.drawLine(20, 50, 200, 50);    // Line  
        g.drawRect(50, 80, 100, 60);    // Rectangle  
        g.drawOval(200, 80, 80, 80);    // Circle  
        g.fillRect(50, 170, 100, 50);    // Filled rectangle  
        g.fillOval(200, 170, 80, 60);    // Filled oval  
    }  
}
```

OUTPUT:



Working with Color and Font Classes in Java (AWT)

Java Color Class

The **Color** class is part of the **Java AWT (Abstract Window Toolkit)** package.

It is used to **create and manipulate colors** in Java graphics.

A color can be created using **RGBA values** or **HSB values**.

- **R – Red**
- **G – Green**
- **B – Blue**
- **A – Alpha (Transparency)**

RGBA Range

- Integer values: **0 to 255**
- Float values: **0.0 to 1.0**

The **alpha value controls transparency**:

- **0 or 0.0 → Fully transparent**
- **255 or 1.0 → Fully opaque**

Constructors of Color Class

1. Color(int red, int green, int blue)

Creates a color using **RGB values**.

Color c = new Color(255, 100, 100);

Example: Light red color

Values must be between **0 – 255**.

2. Color(int rgbValue)

Creates a color using **combined RGB value**.

- Red component → bits **16–23**
- Green component → bits **8–15**
- Blue component → bits **0–7**

3. Color(float red, float green, float blue)

Creates a color using **float values**.

Range: **0.0 – 1.0**

Example:

Color c = new Color(0.5f, 0.4f, 0.3f);

Commonly Used Methods of Color Class

Method	Explanation
brighter()	Creates a brighter version of the color
darker()	Creates a darker version of the color
equals(Object obj)	Checks if two color objects are equal
getBlue()	Returns blue component (0–255)
getGreen()	Returns green component (0–255)
getRed()	Returns red component (0–255)
getColor(String nm)	Finds color from system properties
getTransparency()	Returns transparency mode
getRGB()	Returns RGB value of the color

Setting Color in Graphics

To use color in graphics we use:

```
g.setColor(Color.red);
```

or

```
Color c = new Color(255,100,100);
```

```
g.setColor(c);
```

Color Demonstration Applet

```
import java.applet.Applet;
```

```
import java.awt.*;
```

```
/*  
<applet code="ColorDemo.class" width=300 height=200>  
</applet>  
*/
```

```
public class ColorDemo extends Applet  
{  
    public void paint(Graphics g)  
    {  
        Color c1 = new Color(255,100,100);  
        Color c2 = new Color(100,255,100);  
        Color c3 = new Color(100,100,255);  
        g.setColor(c1);  
        g.drawLine(0,0,100,100);  
        g.drawLine(0,100,100,0);  
        g.setColor(c2);  
        g.drawLine(40,25,250,180);  
        g.setColor(c3);  
        g.drawLine(20,150,200,40);  
        g.setColor(Color.red);  
        g.drawOval(10,10,50,50);  
        g.fillOval(70,90,140,100);  
        g.setColor(Color.blue);
```

```
        g.drawRect(10,10,60,50);
    }
}
```

Font Class in Java

The **Font class** is used to **change the style, size, and type of text** in graphics.

Font Constructor

Font(String name, int style, int size)

Font Styles

- **Font.PLAIN**
- **Font.BOLD**
- **Font.ITALIC**

Example:

```
Font f = new Font("Arial", Font.BOLD, 20);
g.setFont(f);
g.drawString("Hello Java", 50, 50);
```

AWT Classes in Java

Java AWT (Abstract Window Toolkit) is a collection of **classes and APIs used to develop GUI (Graphical User Interface) or window-based applications** in Java.

- AWT components are **platform-dependent**.
- The appearance of components **depends on the operating system**.

- AWT components are **heavyweight** because they use **native OS resources**.

The **java.awt package** provides classes such as:

- Button
- Label
- TextField
- TextArea
- Checkbox
- Choice
- List
- Scrollbar

These classes help in creating **windows, graphics, and event handling**.

Container Class

A **Container** is a component that **can hold other GUI components** like buttons, labels, and text fields.

Classes that extend the **Container class** are called **containers**.

Examples:

- Frame
- Panel
- Dialog
- Window

Containers are used to **organize and display components in a GUI application**.

Types of Containers in AWT

There are **four main types of containers** in AWT.

1. Window

- A **Window** is a container **without borders and menu bars**.
- It is the **top-level container**.
- It must be associated with another container like **Frame** or **Dialog**.

Example:

```
Window w = new Window(new Frame());
```

2. Panel

- A **Panel** is a container used to **group components**.
- It **does not contain a title bar or menu bar**.
- It is commonly used **inside Frame**.

Example:

```
Panel p = new Panel();
```

Components like buttons and text fields can be added to it.

3. Dialog

- A **Dialog** is a container that has **a title and border**.
- It is used to **display messages or input boxes**.
- A Dialog **must be associated with a Frame**.

Example:

```
Dialog d = new Dialog(frame, "Dialog Box");
```

4. Frame

- A **Frame** is the **main window of an AWT application**.
- It contains **title bar, border, and menu bar**.
- It can hold other components like **buttons, labels, text fields, etc.**

Example:

```
Frame f = new Frame("My Application");
```

Event Handling Mechanism in Java

An **event** is a **change in the state of an object**.

Events occur due to **user interaction with GUI components**.

Examples of Events

- Clicking a **button**
- Moving the **mouse**
- Pressing a **keyboard key**
- Selecting an item from a **list**
- **Scrolling** a page

Event Handling

Event Handling is the mechanism that **controls events and defines what action should occur when an event happens**.

The code that executes when an event occurs is called an **Event Handler**.

Java handles events using the **Delegation Event Model**.

Delegation Event Model

The **Delegation Event Model** is the standard mechanism used by Java to **generate and handle events**.

It has **three main participants**:

1. Source

- The **source** is the object where the event occurs.
- It **generates the event**.
- It sends event information to the **listener**.

Examples of source objects

- Button
- TextField
- Checkbox
- List

Example:

If a user clicks a **button**, the button becomes the **event source**.

2. Listener

- The **listener** is also called the **event handler**.
- It is responsible for **processing the event**.
- It waits until it receives an event from the source.
- After receiving the event, it **executes the required action**.

Listeners must **register with the source** to receive event notifications.

Example:

```
button.addActionListener(this);
```

Advantages of Event Handling

1. Separation of logic

The user interface and event processing logic are separated.

2. Efficient processing

Only listeners that register with the source receive event notifications.

3. Flexible design

The interface element delegates the event processing to a separate piece of code.

Steps to Handle an Event in Java

Writing an event-driven program involves **two main steps**.

Step 1: Implement Listener Interface

The listener class must implement the **appropriate event listener interface**.

Example:

```
class MyClass implements ActionListener
```

Step 2: Register the Listener

Register the listener with the **event source**.

Example:

```
button.addActionListener(this);
```

Event Handling Process

- 1. The user performs an action** (e.g., clicks a button).

2. The event object is created automatically.
3. The event object contains information about the event and source.
4. The event object is sent to the registered listener method.
5. The listener processes the event and executes the code.

I will organize your notes clearly for study and exam writing.

Applets, AWT and Event Handling

Different Event Classes

In Java, event classes represent different types of events that occur due to user interaction.

At the root of the event class hierarchy is the **EventObject** class which belongs to the **java.util** package.

It is the superclass for all event classes.

Constructor of EventObject

EventObject(Object src)

- **src** → the object that generated the event.

Methods of EventObject

1. getSource()

Returns the source of the event.

Object getSource()

2. toString()

Returns the **string representation of the event**.

Important Event Classes

Event Class	Description
ActionEvent	Generated when a button is pressed, menu item selected, or list item double-clicked
AdjustmentEvent	Generated when a scroll bar is manipulated
ComponentEvent	Generated when a component is moved, resized, hidden, or shown
ContainerEvent	Generated when a component is added or removed from a container
FocusEvent	Generated when a component gains or loses keyboard focus
InputEvent	Abstract superclass for component input events
ItemEvent	Generated when a checkbox or list item is selected or deselected
KeyEvent	Generated when a key is pressed, released, or typed
MouseEvent	Generated when the mouse is clicked, pressed, released, moved, or dragged
MouseWheelEvent	Generated when the mouse wheel is moved

Event Class	Description
TextEvent	Generated when text in a text field or text area changes
WindowEvent	Generated when a window is opened, closed, activated, or minimized

Event Listener Interfaces

Listeners are created by implementing **interfaces from the java.awt.event package**.

When an event occurs:

1. The **event source** generates an event.
2. The **listener receives the event object**.
3. The **appropriate listener method is executed**.

Common Event Listener Interfaces

Interface	Description
ActionListener	Receives action events
AdjustmentListener	Handles scroll bar adjustment events
ComponentListener	Detects component movement, resizing, hiding
ContainerListener	Detects addition or removal of components
FocusListener	Detects focus gained or lost
ItemListener	Detects selection or deselection of items

Interface	Description
KeyListener	Handles keyboard events
MouseListener	Handles mouse click events
MouseMotionListener	Handles mouse movement and dragging
MouseWheelListener	Handles mouse wheel movement
TextListener	Detects text changes
WindowListener	Handles window events

ActionListener Interface

The **ActionListener** interface handles **action events** such as button clicks.

Method

void actionPerformed(ActionEvent ae)

This method is **called when an action event occurs**.

KeyListener Interface

The **KeyListener** interface handles **keyboard events**.

Methods

void keyPressed(KeyEvent ke)

void keyReleased(KeyEvent ke)

`void keyTyped(KeyEvent ke)`

Example sequence:

If the user presses **A** key:

1. `keyPressed()`
2. `keyTyped()`
3. `keyReleased()`

If the user presses **HOME** key:

1. `keyPressed()`
2. `keyReleased()`

MouseListener Interface

The **MouseListener** interface handles **mouse actions**.

Methods

`void mouseClicked(MouseEvent me)`

`void mouseEntered(MouseEvent me)`

`void mouseExited(MouseEvent me)`

`void mousePressed(MouseEvent me)`

`void mouseReleased(MouseEvent me)`

Explanation:

- **mouseClicked()** → called when mouse is clicked
- **mouseEntered()** → when mouse enters component
- **mouseExited()** → when mouse leaves component
- **mousePressed()** → when mouse button is pressed
- **mouseReleased()** → when mouse button is released

Java AWT Program to demonstrate MouseListener Handling

```
import java.awt.*;
import java.awt.event.*;
public class Main {
    public static void main(String[] args){
        // Create an instance of frame
        Frame f = new Frame("MouseListener Demo");
        // Create a label
        Label l = new Label("Welcome to
GeeksforGeeks!");
        // Set the properties of label
        l.setBounds(80, 100, 220, 30);
        l.setFont(new Font("Serif", Font.BOLD, 18));
        l.setForeground(Color.GREEN);

        // Add MouseListener to the label with different
        methods
        l.addMouseListener(new MouseListener() {

            public void mouseClicked(MouseEvent e){
                l.setText("Mouse Clicked");
            }
        })
    }
}
```

```
        public void mouseEntered(MouseEvent e){
            l.setText("Mouse Entered");
        }
        public void mouseExited(MouseEvent e){
            l.setText("Mouse Exited");
        }
        public void mousePressed(MouseEvent e){
            l.setText("Mouse Pressed");
        }
        public void mouseReleased(MouseEvent e){
            l.setText("Mouse Released");
        }
    });
    // Add label to the frame
    f.add(l);
    // Set the properties of frame
    f.setLayout(null);
    f.setSize(400, 300);
    f.setVisible(true);
}
}
```

A screenshot of an IDE window showing a Java file named 'Main.java'. The code defines a 'Main' class with a 'main' method. Inside the 'main' method, it creates a 'Frame' titled 'MouseListener Demo' and a 'Label' with the text 'Welcome to GeeksforGeeks!'. The label is then added to the frame. The IDE interface includes a sidebar with 'Main.java' selected, a toolbar with 'Run' and 'Debug' buttons, and a bottom panel with tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', 'PORTS', and 'DEBUG CONSOLE'. The 'TERMINAL' tab is active, showing a command prompt path: 'PS E:\Web Dev Projects\GF6\Java\AWT>'.

```
1 import java.awt.*;
2 import java.awt.event.*;
3
4 public class Main {
5     public static void main(String[] args) {
6         // Create an instance of frame
7         Frame f = new Frame(title:"MouseListener Demo");
8         // Create a Label
9         Label l = new Label(text:"Welcome to GeeksforGeeks!");
10        // Set the properties of label
11        l.setBounds(50, 50, 100, 30);
12    }
13 }
```

Java AWT Label

```
import java.awt.*;
```

```
import java.awt.event.WindowAdapter;
```

```
import java.awt.event.WindowEvent;
```

```
// Driver Class
```

```
class Main {
```

```
    // Main Function
```

```
    public static void main(String[] args)
```

```
    {
```

```
        // Declaring Frame
```

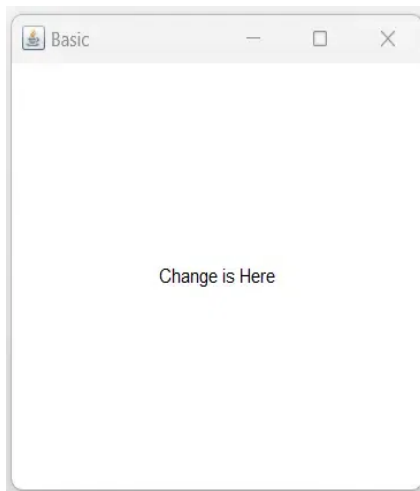
```
        Frame frame = new Frame("Basic");
```

```
        // All the components will be
```

```
        // inserted in this frame
```

```
// Declaring Label
Label label = new Label("Change is Here");
// Setting the Alignment to CENTER
label.setAlignment(Label.CENTER);
// Adding Label into Frame
frame.add(label);
// Setting Frame Size
frame.setSize(500, 500);
// Setting Visibility of Frame
frame.setVisible(true);
// Using WindowListener for closing the window
frame.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
});
}}
```

Output:



Java Button (AWT)

A **Button** is a GUI component in Java AWT that is used to **trigger an action when the user clicks it**.

The **Button** class is available in the **java.awt** package.

When a button is clicked, it generates an **ActionEvent**, which can be handled using the **ActionListener** interface.

Source code:

```
import java.awt.Button;
import java.awt.Frame;
// Driver Class
public class Main {
    // main function
    public static void main(String[] args)
    {
        // Create a frame
        Frame frame = new Frame("AWT Button Example");
```

```
// Create a button
Button button = new Button("Click Me!");

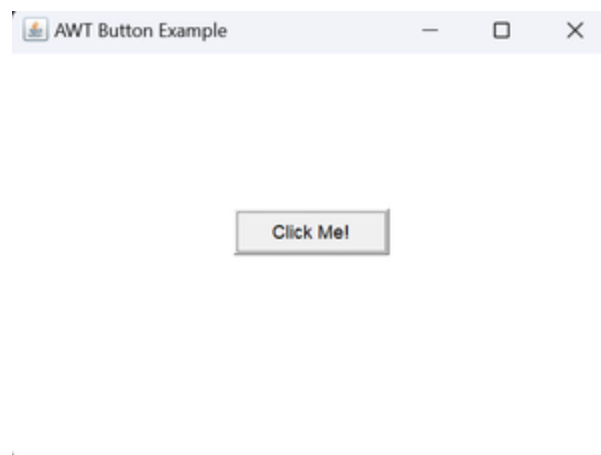
// Set the button position on the frame
button.setBounds(150, 130, 100, 30);

// Add the button to the frame
frame.add(button);

// Set the frame size and layout
frame.setSize(400, 300);
frame.setLayout(null);

// Set the frame visibility to true
frame.setVisible(true);
}
}
```

Output:



Java CheckBox (AWT)

A **CheckBox** is a GUI component in Java AWT that allows the user to **select or deselect an option**.

It is used when **multiple options can be selected at the same time**.

The **Checkbox** class is available in the **java.awt** package.

When a checkbox is selected or deselected, it generates an **ItemEvent**, which can be handled using the **ItemListener** interface.

Source code:

```
import java.awt.*;

public class one {

    //Constructor to initialize

    public one() {

        // Creating a Frame

        Frame frame = new Frame("Simple Checkbox
Example");

        // Creating a Checkbox with the label "Agree to terms
and conditions"

        Checkbox checkbox = new Checkbox("Agree to
geeksforgeeks terms and conditions");

        // Setting Checkbox position

        checkbox.setBounds(50, 50, 250, 30);

        // Adding Checkbox to the frame

        frame.add(checkbox);
```

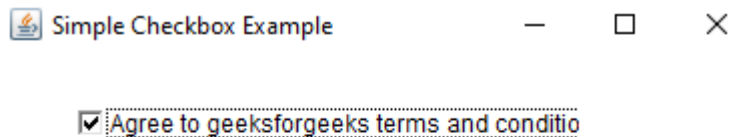
```
// Setting Frame size
frame.setSize(400, 200);

// Setting Layout to null
frame.setLayout(null);

// Making the frame visible
frame.setVisible(true);
}

public static void main(String[] args) {
    new one();
}
}
```

Output:



Java List (AWT)

A **List** in Java AWT is a GUI component that displays a **scrollable list of items** from which the user can **select one or more items**.

The **List** class is available in the **java.awt** package.

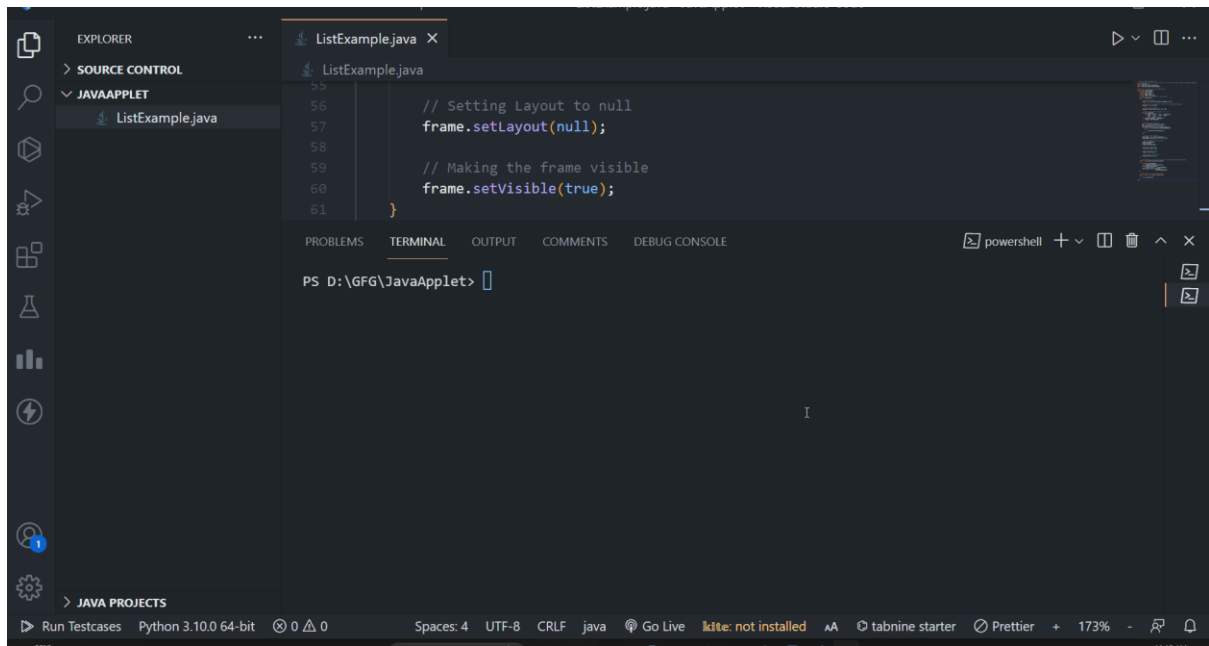
It is commonly used when the program needs to **present multiple options to the user**.

Source code:

```
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
// Class that is used as the main application
public class ListExample {
    private Frame frame;
    private List langList;
    private Button btn;
    private Label selectLang;
    public ListExample(){
        // Creating a Frame
        frame = new Frame("Programming Languages List");
        langList = new List(10);
        // Setting List Postition
        langList.setBounds(100, 100, 150, 150);
        // Adding programming languages to the list
        String[] languages
            = { "Java", "Python", "C++", "JavaScript",
              "Ruby", "C#",   "PHP", "Swift",
              "Go",  "Kotlin" };
        for (String language : languages) {
            langList.add(language);
```

```
}  
  
// Setting Button Position  
btn = new Button("Display Selected");  
btn.setBounds(100, 270, 100, 30);  
btn.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e)  
    {  
        displaySelectedLanguages();  
    }  
});  
  
// Setting Label Position  
selectLang = new Label("Languages:");  
selectLang.setBounds(100, 310, 150, 20);  
  
// Adding Components to the frame  
frame.add(langList);  
frame.add(btn);  
frame.add(selectLang);  
  
  
// Setting Frame Size  
frame.setSize(400, 400);  
  
  
// Setting Layout to null  
frame.setLayout(null);
```

```
// Making the frame visible
frame.setVisible(true);
}
// This method will be used to display the selected
// programming languages.
public void displaySelectedLanguages(){
    String[] selectedItems = langList.getSelectedItems();
    String selectedLanguages = String.join(", ",
selectedItems);
    selectLang.setText("Languages: "+ selectedLanguages);
}
// Main method to run the application
public static void main(String[] args){
    new ListExample();
}
}
```



The screenshot shows an IDE interface with a dark theme. On the left, the Explorer panel shows a project named 'JAVAAPPLET' containing a file 'ListExample.java'. The main editor displays the code for 'ListExample.java' with line numbers 55 to 61. The code is as follows:

```
55  
56 // Setting Layout to null  
57 frame.setLayout(null);  
58  
59 // Making the frame visible  
60 frame.setVisible(true);  
61 }
```

Below the editor, the TERMINAL panel is active, showing a PowerShell prompt at 'PS D:\GFG\JavaApplet>'. The status bar at the bottom indicates 'Spaces: 4', 'UTF-8', 'CRLF', 'java', and other settings.

thank you

- Lohith Kumar